



RÉPUBLIQUE TUNISIENNE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITÉ DE TUNIS EL MANAR

FACULTÉ DES SCIENCES DE TUNIS
DÉPARTEMENT DES SCIENCES DE L'INFORMATIQUE

Rapport de Stage d'Été

Filière : Génie Logiciel

PRLens

Agent IA de revue de code automatisée
pour les Pull Requests GitHub

Réalisé par :

Ismail Mechkene

Encadré par :

M. Abdelkhalek Baccari

Réalisé au sein de :



Année Universitaire : 2025–2026

Remerciements

Au terme de ce travail, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage.

Je remercie chaleureusement l'entreprise **Smartovate** pour m'avoir accueilli au sein de son équipe et m'avoir offert l'opportunité de travailler sur un projet aussi formateur, à la croisée de l'intelligence artificielle générative et de l'ingénierie logicielle.

J'adresse mes remerciements les plus sincères à mon encadrant, M. Abdelkhalek Baccari, pour sa disponibilité, la pertinence de ses conseils et la rigueur qu'il a su transmettre tout au long de ce projet. Ses orientations ont été déterminantes dans la structuration de la démarche adoptée, depuis l'analyse du besoin jusqu'à la validation quantitative de la solution.

Mes remerciements s'adressent également à la Faculté des Sciences de Tunis ainsi qu'à toute l'équipe pédagogique du Département des Sciences de l'Informatique, pour la qualité de la formation dispensée durant ce cursus, qui a constitué le socle des compétences mobilisées dans ce projet.

Enfin, je remercie ma famille et mes proches pour leur soutien constant et leurs encouragements.

Table des matières

Remerciements	i
Table des figures	v
Liste des tableaux	vi
Liste des abréviations	vii
Introduction générale	1
1 Cadre général du projet	3
Introduction	4
1.1 Présentation de l'organisme d'accueil	4
1.1.1 Mission et domaines d'expertise	4
1.2 Cadre du projet	4
1.3 Problématique	5
1.4 Étude de l'existant	6
1.4.1 Analyseurs statiques classiques	6
1.4.2 Assistants de revue fondés sur un LLM	6
1.4.3 Scanners de sécurité dédiés	6
1.4.4 Synthèse comparative	6
1.5 Solution proposée	7
1.6 Méthodologie adoptée	8
1.6.1 Définition d'une méthodologie Agile	8
1.6.2 Comparaison entre méthodes classiques et méthodes agiles	8
1.6.3 Synthèse pour le choix de Scrum	9
Conclusion	10
2 Analyse et spécification des besoins	11
Introduction	12
2.1 Identification des acteurs	12
2.2 Besoins fonctionnels	12
2.3 Besoins non fonctionnels	13
2.4 Diagramme de cas d'utilisation global	14
2.5 Description textuelle des cas d'utilisation	15
2.6 Diagrammes de séquence	18

2.6.1	Authentification et connexion d'un dépôt	18
2.6.2	Cycle interne d'une revue	19
2.7	Backlog produit	19
2.8	Planification des itérations	20
	Conclusion	21
3	Conception	22
	Introduction	23
3.1	Architecture globale du système	23
3.2	Architecture technique	23
3.2.1	Le moteur de revue	24
3.2.2	Le serveur webhook et l'API REST	24
3.2.3	Le tableau de bord	24
3.3	Diagramme de déploiement	24
3.4	Diagramme d'activité — le cycle de revue	25
3.5	Diagramme de classes	26
3.6	Conception de la base de données	27
3.7	Maquettes de l'interface	28
3.8	Algorithmes et règles de décision	29
3.8.1	Analyse parallèle des fichiers	29
3.8.2	Calcul du score et du verdict	30
3.8.3	Gestion des échecs partiels	30
3.9	Sécurité et gestion des accès	30
3.9.1	Mesures de sécurité	30
3.9.2	Espace d'administration et moindre privilège	31
3.10	Choix technologiques et justifications	32
	Conclusion	33
4	Réalisation	34
	Introduction	35
4.1	Environnement de développement	35
4.2	Itération 1 — Moteur de revue et authentification GitHub (semaines 1–2)	35
4.2.1	Objectifs	35
4.2.2	Réalisation	36
4.2.3	Résultats	36
4.3	Itération 2 — Mode webhook et persistance (semaines 3–4)	36
4.3.1	Objectifs	37
4.3.2	Réalisation	37
4.3.3	Résultats	37
4.4	Itération 3 — Tableau de bord et authentification OAuth (semaines 5–6)	37

4.4.1	Objectifs	37
4.4.2	Réalisation	37
4.5	Itération 4 — Historique, administration et quotas (semaines 7–9) . . .	38
4.5.1	Objectifs	38
4.5.2	Réalisation	38
4.5.3	Résultats	38
4.6	Tests et validation	39
4.6.1	Stratégie de tests	39
4.6.2	Résultats des tests unitaires	39
4.6.3	Harnais d'évaluation	39
4.7	Déploiement en production	41
	Conclusion	42
	Conclusion générale et perspectives	43
	Webographie	45
	Appendices	46
A	Exemple de fichier de configuration <code>.aireviewer.yml</code>	47
B	Exemple de commentaire généré par PRLens	48
C	Extrait du modèle de données (SQLAlchemy)	49
D	Liste complète des variables d'environnement	50
E	Liste complète des points d'accès de l'API REST	51

Table des figures

1.1	Logo de l'entreprise Smartovate	4
1.2	Méthodologie traditionnelle <i>versus</i> méthodologie Agile	8
2.1	Diagramme de cas d'utilisation global du système PRLens	15
2.2	Diagramme de séquence — connexion, configuration et revue d'une Pull Request	18
2.3	Diagramme de séquence — cycle interne d'une revue	19
3.1	Architecture globale du pipeline de revue PRLens	23
3.2	Diagramme de déploiement de PRLens en production	25
3.3	Diagramme d'activité du cycle de revue	26
3.4	Diagramme de classes du moteur de revue	27
3.5	Modèle de données relationnel (extrait des modèles SQLAlchemy)	27
3.6	Maquette de la page d'accueil publique	28
3.7	Maquette de l'interface principale du tableau de bord	29
3.8	Maquette de la page de détail d'un dépôt	29
4.1	Commentaire en ligne généré automatiquement sur une Pull Request	36
4.2	Espace d'administration	38
4.3	Hébergement du serveur FastAPI sur Railway	42
4.4	Base de données PostgreSQL hébergée sur Supabase	42

Liste des tableaux

1.1	Positionnement de PRLens par rapport aux catégories d'outils existantes	7
1.2	Comparaison synthétique des approches classiques et agiles	9
1.3	Planification des sprints du projet	9
2.1	Acteurs interagissant avec le système PRLens	12
2.2	Besoins fonctionnels du système PRLens	13
2.3	Besoins non fonctionnels du système	14
2.4	Description textuelle du cas d'utilisation « S'authentifier »	16
2.5	Description textuelle du cas d'utilisation « Connecter un dépôt » . . .	16
2.6	Description textuelle du cas d'utilisation « Recevoir une revue automa- tique »	17
2.7	Backlog produit du projet PRLens	20
2.8	Planification des itérations du projet	21
3.1	Règles de décision du verdict de revue	30
3.2	Pile technologique du projet et justification des choix	32
4.1	Environnement de développement et technologies utilisées	35
4.2	Couverture de tests par module	40
4.3	Résultats du harnais d'évaluation	41
D.1	Variables d'environnement du serveur PRLens	50
E.1	Points d'accès exposés par le serveur FastAPI	52

Liste des abréviations

API	<i>Application Programming Interface</i> — interface de programmation applicative
CI/CD	<i>Continuous Integration / Continuous Deployment</i> — intégration et déploiement continus
CORS	<i>Cross-Origin Resource Sharing</i> — partage de ressources entre origines
DTO	<i>Data Transfer Object</i> — objet de transfert de données
HMAC	<i>Hash-based Message Authentication Code</i> — code d'authentification de message
IA	Intelligence Artificielle
JSON	<i>JavaScript Object Notation</i>
JWT	<i>JSON Web Token</i> — jeton d'authentification signé
LLM	<i>Large Language Model</i> — grand modèle de langage
ORM	<i>Object-Relational Mapping</i> — correspondance objet-relationnel
OWASP	<i>Open Worldwide Application Security Project</i>
PR	<i>Pull Request</i> — demande d'intégration de modifications
REST	<i>Representational State Transfer</i>
SGBD	Système de Gestion de Base de Données
SPA	<i>Single Page Application</i> — application web monopage
SQL	<i>Structured Query Language</i>
UML	<i>Unified Modeling Language</i>
YAML	<i>YAML Ain't Markup Language</i> — format de fichier de configuration

Introduction générale

La revue de code (*code review*) est aujourd’hui une étape incontournable du cycle de développement logiciel : elle conditionne la qualité, la sécurité et la maintenabilité d’une base de code, et constitue le principal garde-fou avant l’intégration de tout changement dans une branche principale. Sur la plateforme GitHub, ce processus prend la forme de la *Pull Request* (PR), un mécanisme qui propose un ensemble de modifications, ouvre une discussion et exige généralement l’approbation d’un ou plusieurs relecteurs humains avant fusion.

Or cette relecture humaine se heurte à des limites bien connues : elle est chronophage, elle dépend fortement de la disponibilité et de l’attention du relecteur, et elle est sujette à l’inconstance — deux relecteurs, voire un même relecteur à deux moments différents, ne détectent pas systématiquement les mêmes problèmes. Dans des équipes où le volume de Pull Requests est important, ce goulot d’étranglement ralentit le rythme de livraison et laisse régulièrement passer des défauts évitables : secrets codés en dur, injections, fonctions trop complexes, absence de gestion d’erreurs.

L’essor récent des grands modèles de langage (*Large Language Models*, LLM) change les termes du problème. Entraînés sur de très larges corpus incluant du code source, ces modèles sont capables d’inférer l’intention d’un extrait de code, d’y identifier des anomalies et de proposer des corrections formulées en langage naturel — une capacité qui dépasse ce que permettent les analyseurs statiques fondés sur des règles syntaxiques fixes.

C’est dans ce contexte qu’a été conçu **PRLens**, objet du présent rapport : un système de revue de code automatisée qui s’appuie sur un LLM pour analyser le contenu exact d’une Pull Request et produire, sans intervention humaine, des commentaires en ligne, un score de qualité chiffré, un verdict de revue formel (Approuver / Demander des changements / Commenter) et des étiquettes de classification. PRLens ne remplace pas la revue humaine : il agit comme un premier filtre systématique, rapide et reproductible, qui traite les problèmes les plus fréquents et laisse au relecteur humain le soin de juger des aspects fonctionnels et architecturaux de plus haut niveau.

Le projet se décompose en deux volets complémentaires. Le premier est le **moteur de revue**, déployable de deux manières : comme étape d’un pipeline GitHub Actions (sans serveur à héberger), ou comme serveur *webhook* permanent installé une fois pour tous les dépôts d’une organisation. Le second volet est un **tableau de bord web** qui permet à un utilisateur de se connecter avec son compte GitHub, de connecter ses

dépôts en un clic, de configurer le comportement de la revue par dépôt, et de consulter l'historique et les tendances de qualité mesurées au fil du temps.

Ce rapport s'organise en quatre chapitres. Le premier situe le cadre général du projet : l'organisme d'accueil, la problématique, l'étude des solutions existantes, la solution proposée et la méthodologie de gestion adoptée. Le deuxième est consacré à l'analyse et à la spécification des besoins, modélisés à l'aide des diagrammes UML. Le troisième détaille la conception de la solution : architecture globale et technique, modélisation des données, interfaces et algorithmes. Le quatrième décrit la réalisation itération par itération, ainsi que la validation du système par les tests unitaires et par un protocole d'évaluation quantitatif. Une conclusion générale dresse enfin le bilan du travail accompli et ouvre sur des perspectives d'évolution.

CHAPITRE 1

Cadre général du projet

Sommaire du chapitre

1. Introduction
2. Présentation de l'organisme d'accueil
 - Mission et domaines d'expertise
 - Organigramme
3. Cadre du projet
4. Problématique
5. Étude de l'existant
 - Analyseurs statiques classiques
 - Assistants de revue fondés sur un LLM
 - Scanners de sécurité dédiés
 - Synthèse comparative
6. Solution proposée
7. Méthodologie adoptée
 - Définition d'une méthodologie Agile
 - Comparaison entre méthodes classiques et méthodes agiles
 - Synthèse pour le choix de Scrum
8. Conclusion

Introduction

Ce chapitre pose le cadre général dans lequel s'inscrit le projet. Il présente l'organisme d'accueil, dégage la problématique à partir d'une analyse du terrain, examine les solutions déjà disponibles sur le marché, expose la solution proposée et justifie enfin la méthodologie de gestion de projet retenue pour la conduite du développement.

1.1 Présentation de l'organisme d'accueil

1.1.1 Mission et domaines d'expertise

Smartovate est une entreprise spécialisée dans le développement de solutions logicielles fondées sur les technologies de l'intelligence artificielle et du cloud computing. Elle conçoit des applications adaptées aux besoins métier de ses clients, en s'appuyant sur des approches modernes orientées automatisation, traitement de données et intelligence artificielle.

L'entreprise intervient principalement dans les domaines suivants :

- Développement web et mobile ;
- Intelligence artificielle et agents autonomes ;
- Services cloud et architectures distribuées ;
- Internet des objets (IoT).

Grâce à son expertise technique, Smartovate accompagne ses clients tout au long du cycle de développement logiciel, depuis l'analyse des besoins jusqu'à la mise en production des solutions.



Figure 1.1 – Logo de l'entreprise Smartovate

1.2 Cadre du projet

Le présent projet a été réalisé dans le cadre du stage de la filière **Génie Logiciel** du Département des Sciences de l'Informatique de la Faculté des Sciences de Tunis (FST). Il s'inscrit dans la catégorie des projets à caractère applicatif portant sur les nouvelles technologies, avec pour terrain d'application concret l'automatisation d'un processus de développement logiciel à l'aide de l'intelligence artificielle générative.

L’encadrement a porté à la fois sur la méthodologie de conduite de projet — structuration du travail, jalons, livrables — et sur les choix techniques structurants du système : architecture logicielle, intégration avec les services externes (GitHub, Azure OpenAI) et exigences de qualité (couverture de tests, évaluation de la pertinence des sorties du modèle).

Il s’agit d’un **stage d’été**, réalisé du **1^{er} juin au 31 juillet 2026**, soit une durée de deux mois. Cette période a été découpée en quatre itérations d’environ deux semaines chacune, selon la planification présentée au §1.6.3.

1.3 Problématique

L’analyse du terrain met en évidence plusieurs constats récurrents au sein des équipes de développement :

1. **Délai de premier retour** — un développeur qui attend plusieurs heures, voire plusieurs jours, une première relecture perd le contexte mental de son changement et voit sa productivité baisser.
2. **Manque de systématique** — certaines classes de problèmes (secrets codés en dur, injections SQL, fonctions démesurément complexes) devraient être détectées à chaque fois, indépendamment de la charge de travail ou de la fatigue du relecteur.
3. **Absence de traçabilité** — l’équipe ne dispose d’aucun historique exploitable permettant de savoir quel dépôt est le plus souvent en cause, quel type de problème revient, ou si la qualité du code progresse dans le temps.
4. **Coût d’adoption élevé** — une solution qui exige une reconfiguration lourde de l’infrastructure existante peine à être adoptée par des équipes déjà très sollicitées.

La problématique centrale se formule ainsi :

Comment fournir, de manière automatique et systématique, une première revue de qualité professionnelle sur chaque Pull Request GitHub, sans imposer de contrainte d’infrastructure lourde et en laissant à l’équipe le contrôle fin du comportement de cette revue ?

Cette problématique se décompose en quatre défis techniques :

1. Analyser un *diff* de manière suffisamment fine pour produire des commentaires **sur la ligne exacte concernée**, et non une observation générale peu actionnable.
2. Produire un verdict et un score **reproductibles** à partir d’une source par nature non déterministe, afin que le résultat reste exploitable en pratique (étiquetage, blocage de fusion).

3. Permettre à chaque équipe de **personnaliser** le comportement de la revue — langages cibles, fichiers exclus, seuils de score, relecteurs à affecter — sans réécrire de code.
4. S'intégrer nativement dans **deux contextes d'usage** distincts : une équipe qui préfère un pipeline sans état, et une équipe qui préfère une installation centralisée sans configuration par dépôt.

1.4 Étude de l'existant

Avant de proposer notre solution, nous analysons les outils d'aide à la revue de code disponibles sur le marché. Ils se répartissent en trois familles.

1.4.1 Analyseurs statiques classiques

Les outils tels que SonarQube, ESLint ou Pylint analysent le code source à partir de règles syntaxiques fixes et d'un arbre de syntaxe abstraite. Ils sont rapides, entièrement déterministes et bien intégrés aux chaînes d'intégration continue. En revanche, ils ne raisonnent pas sur l'*intention* du code : une clé d'API codée en dur n'est détectée que si un motif la décrit explicitement, et aucun de ces outils ne formule de suggestion de correction rédigée en langage naturel.

1.4.2 Assistants de revue fondés sur un LLM

Une seconde famille, apparue plus récemment, s'appuie directement sur les grands modèles de langage : CodeRabbit ou la fonction de revue de Pull Request de GitHub Copilot en sont les représentants les plus visibles. Ces solutions apportent la compréhension sémantique qui manque aux analyseurs statiques, mais elles sont proposées exclusivement en mode SaaS : le code est transmis à un service tiers, le modèle de scoring n'est pas documenté, et le comportement de la revue n'est configurable que dans les limites prévues par l'éditeur.

1.4.3 Scanners de sécurité dédiés

Des outils comme Snyk Code ou Semgrep se concentrent sur la seule dimension sécurité, avec une base de règles maintenue et un taux de faux positifs maîtrisé. Leur périmètre est cependant volontairement étroit : ils ne se prononcent ni sur la qualité générale du code, ni sur ses performances, ni sur sa documentation, et doivent donc être complétés par d'autres outils.

1.4.4 Synthèse comparative

Le tableau 1.1 positionne PRLens par rapport à ces trois familles.

Critère	Analyse statique	Assistants LLM (SaaS)	Scanners sécurité	PRLens
Compréhension sémantique	Non	Oui	Partielle	Oui
Commentaires sur la ligne exacte	Oui	Oui	Oui	Oui
Périmètre d'analyse	Qualité, style	Variable	Sécurité	5 dimensions
Score et verdict configurables	Partiel	Non	Non	Oui
Auto-hébergement	Oui	Non	Partiel	Oui
Deux modes de déploiement	Non	Non	Non	Oui
Historique et tendances	Oui	Partiel	Oui	Oui

Tableau 1.1 – Positionnement de PRLens par rapport aux catégories d'outils existantes

1.5 Solution proposée

Pour répondre à la problématique identifiée, nous proposons **PRLens**, un agent de revue de code automatisée doté des caractéristiques suivantes :

- **Analyse multidimensionnelle** — chaque fichier modifié est examiné sous cinq angles : sécurité, qualité, performance, style et documentation.
- **Commentaires en ligne** — les constats sont publiés sur la ligne exacte du *diff* concernée, accompagnés d'une suggestion de correction.
- **Score et verdict formels** — un score de 0 à 100 est calculé par pénalités cumulées, dont est dérivé un verdict GitHub natif (*Approve*, *Request changes*, *Comment*).
- **Deux modes de déploiement** — une étape de pipeline GitHub Actions sans serveur à héberger, ou une application GitHub installée une seule fois pour tous les dépôts d'une organisation, les deux partageant le même moteur de revue.
- **Configuration par dépôt** — langages cibles, motifs de fichiers exclus, seuil de sévérité, seuils de verdict et affectation automatique de relecteurs, réglables depuis le tableau de bord ou via un fichier `.aireviewer.yml` versionné dans le dépôt.
- **Tableau de bord web** — connexion par OAuth GitHub, connexion des dépôts en un clic, historique des revues et tendances de qualité.
- **Espace d'administration** — une vue en lecture seule de l'état global du déploiement, réservée à l'exploitant du service.

- **Validation quantitative** — un harnais d'évaluation hors ligne mesure la pertinence réelle des revues produites, au-delà des seuls tests unitaires.

1.6 Méthodologie adoptée

La complexité d'un projet mêlant intégration de services externes, appels à un modèle de langage et développement d'une interface web exige une méthodologie de gestion rigoureuse. La multiplicité des composants et l'évolution continue des exigences fonctionnelles rendent indispensable l'adoption d'un cadre de travail structuré, capable de s'adapter aux changements tout en maintenant une livraison régulière de valeur.

1.6.1 Définition d'une méthodologie Agile

La méthodologie Agile est une approche de gestion de projet fondée sur un développement itératif et incrémental, favorisant la collaboration et l'adaptation continue aux évolutions des besoins. Contrairement aux méthodes traditionnelles, les approches Agile reposent sur des cycles de développement courts, appelés *sprints*, permettant de concevoir, développer et tester progressivement les fonctionnalités du système.

1.6.2 Comparaison entre méthodes classiques et méthodes agiles

Les méthodes classiques, telles que le cycle en V ou le modèle en cascade, reposent sur une planification exhaustive en amont et une exécution séquentielle des phases : analyse, conception, développement puis tests. Cette rigidité les rend peu adaptées aux projets dont les exigences évoluent en cours de route. À l'inverse, les méthodes agiles découpent le projet en itérations courtes et livrent des incréments fonctionnels réguliers, permettant d'intégrer les retours à chaque cycle.

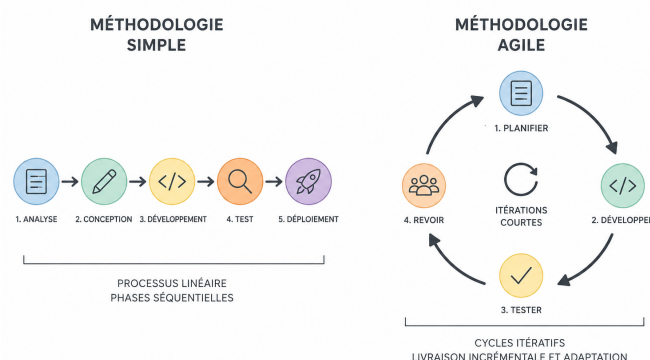


Figure 1.2 – Méthodologie traditionnelle *versus* méthodologie Agile

Le tableau 1.2 synthétise les principales différences entre les deux approches.

Critère	Classique (cascade)	Agile
Planification	Rigide, définie en amont	Itérative, adaptative
Livraison	En fin de projet	Incrémentale (sprints)
Retours	Tardifs	Continus
Gestion du changement	Difficile	Encouragée
Documentation	Exhaustive	Suffisante
Visibilité sur l'avancement	Faible en cours de projet	Élevée à chaque sprint

Tableau 1.2 – Comparaison synthétique des approches classiques et agiles

1.6.3 Synthèse pour le choix de Scrum

Le développement de PRLens repose sur l'intégration de plusieurs composants relativement indépendants — moteur de revue, serveur webhook, tableau de bord — dont chacun peut être livré et validé séparément. Parmi les méthodologies Agile existantes, **Scrum** a été retenu pour son découpage en cycles courts, qui permet de développer, tester et valider progressivement chaque composant.

Ce choix présente plusieurs avantages pour le projet :

- une meilleure organisation des tâches et des fonctionnalités ;
- une adaptation continue aux évolutions techniques et fonctionnelles ;
- un suivi régulier de l'avancement ;
- une livraison progressive des composants développés, chaque sprint se clôturant par une exécution complète de la suite de tests.

La période de stage, du 1^{er} juin au 31 juillet 2026, a été découpée en quatre sprints d'environ deux semaines, comme indiqué au tableau 1.3.

Sprint	Période	Objectifs
Sprint 1	Semaines 1–2 (1–14 juin)	Moteur de revue : récupération de la Pull Request, analyse par LLM, publication des commentaires, authentification GitHub.
Sprint 2	Semaines 3–4 (15–28 juin)	Mode webhook : serveur FastAPI, vérification de signature, persistance des revues en base.
Sprint 3	Semaines 5–6 (29 juin – 12 juillet)	Tableau de bord : authentification OAuth GitHub, connexion et configuration des dépôts.
Sprint 4	Semaines 7–9 (13–31 juillet)	Historique et statistiques, espace d'administration, limite d'utilisation, déploiement en production.

Tableau 1.3 – Planification des sprints du projet

Conclusion

Ce chapitre a posé le cadre général du projet en présentant l'organisme d'accueil, la problématique identifiée et la solution PRLens proposée pour y répondre, ainsi que la méthodologie itérative adoptée pour la conduite du développement. Le chapitre suivant détaille l'analyse et la spécification des besoins fonctionnels et non fonctionnels du système.

CHAPITRE 2

Analyse et spécification des besoins

Sommaire du chapitre

1. Introduction
2. Identification des acteurs
3. Besoins fonctionnels
4. Besoins non fonctionnels
5. Diagramme de cas d'utilisation global
6. Description textuelle des cas d'utilisation
7. Diagrammes de séquence
8. Backlog produit
9. Planification des itérations
10. Conclusion

Introduction

Ce chapitre est consacré à l’analyse et à la spécification des besoins du système PRLens. Nous identifions d’abord les acteurs qui interagissent avec le système, puis nous détaillons les besoins fonctionnels et non fonctionnels. Ces besoins sont ensuite modélisés à l’aide des diagrammes UML — cas d’utilisation et séquence — avant la présentation du backlog produit et de la planification des itérations.

2.1 Identification des acteurs

Les acteurs sont les entités externes, humaines ou logicielles, qui interagissent directement avec le système. Le tableau 2.1 recense ceux identifiés pour PRLens.

Acteur	Rôle et description
Développeur	Auteur d’une Pull Request sur un dépôt connecté. Il ne manipule pas directement PRLens : il en reçoit le résultat sous forme de commentaires et de verdict sur sa Pull Request.
Utilisateur	Le propriétaire d’un ou plusieurs dépôts, authentifié sur le tableau de bord. Il connecte ses dépôts, configure leurs paramètres de revue et consulte l’historique.
Administrateur	L’exploitant du service. Il conserve un compte utilisateur ordinaire et dispose en plus d’un accès en lecture seule à l’état global du déploiement.
GitHub	Système externe. Il émet les événements de Pull Request, fournit le contenu des <i>diffs</i> et reçoit les commentaires, étiquettes et verdicts publiés.
Service LLM	Système externe (Azure OpenAI). Il reçoit le contenu d’un fichier modifié et retourne les constats d’analyse au format JSON.

Tableau 2.1 – Acteurs interagissant avec le système PRLens

2.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions que le système doit être capable d’exécuter. Le tableau 2.2 recense les besoins identifiés, répartis entre le moteur de revue et le tableau de bord.

ID	Fonctionnalité	Description
BF01	Analyse d'une Pull Request	Le système doit analyser le <i>diff</i> de chaque Pull Request ouverte ou mise à jour sur un dépôt connecté.
BF02	Commentaires en ligne	Le système doit publier ses constats sur la ligne exacte concernée, classés par type et par sévérité.
BF03	Score et verdict	Le système doit calculer un score de qualité de 0 à 100 et en dériver un verdict de revue formel.
BF04	Étiquetage automatique	Le système doit appliquer à la Pull Request des étiquettes reflétant le résultat de la revue.
BF05	Affectation de relecteurs	Le système doit pouvoir affecter automatiquement un relecteur humain selon le type de problème détecté.
BF06	Configuration par dépôt	Le système doit permettre de régler par dépôt les langages cibles, les fichiers exclus et les seuils de décision.
BF07	Authentification	Le système doit permettre à l'utilisateur de s'authentifier via son compte GitHub (OAuth 2.0).
BF08	Connexion d'un dépôt	Le système doit permettre de connecter ou de déconnecter un dépôt depuis l'interface, en un clic.
BF09	Suspension d'un dépôt	Le système doit permettre de suspendre temporairement la revue d'un dépôt sans perdre son historique.
BF10	Historique et tendances	Le système doit permettre de consulter l'historique des revues et l'évolution du score dans le temps.
BF11	Supervision globale	Le système doit fournir à l'administrateur une vue agrégée de l'état du déploiement.

Tableau 2.2 – Besoins fonctionnels du système PRLens

2.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les critères de qualité et les contraintes techniques du système.

Catégorie	Description
Performance	L'analyse d'une Pull Request touchant plusieurs fichiers doit être parallélisée, afin que le temps de réponse ne croisse pas linéairement avec le nombre de fichiers modifiés.
Fiabilité	L'échec de l'analyse d'un fichier ne doit pas faire échouer la revue entière : le système doit distinguer un échec partiel d'un échec total et le signaler explicitement.
Sécurité	Les événements webhook doivent être authentifiés par signature HMAC. Les jetons d'accès doivent être à durée de vie courte et jamais exposés au client.
Reproductibilité	À constats identiques, le score et le verdict produits doivent être identiques : la logique de décision doit rester entièrement déterministe.
Maintenabilité	Le moteur de revue doit être couvert par une suite de tests automatisés s'exécutant hors ligne, sans consommer de quota d'API externe.
Portabilité	Le moteur doit s'exécuter à l'identique dans un conteneur Docker (mode Actions) et dans un serveur permanent (mode webhook).
Utilisabilité	La connexion d'un dépôt doit se faire sans édition de fichier de configuration ni manipulation en ligne de commande.

Tableau 2.3 – Besoins non fonctionnels du système

2.4 Diagramme de cas d'utilisation global

La figure 2.1 présente le diagramme de cas d'utilisation global du système, faisant apparaître les cas rattachés à chacun des acteurs identifiés au tableau 2.1.

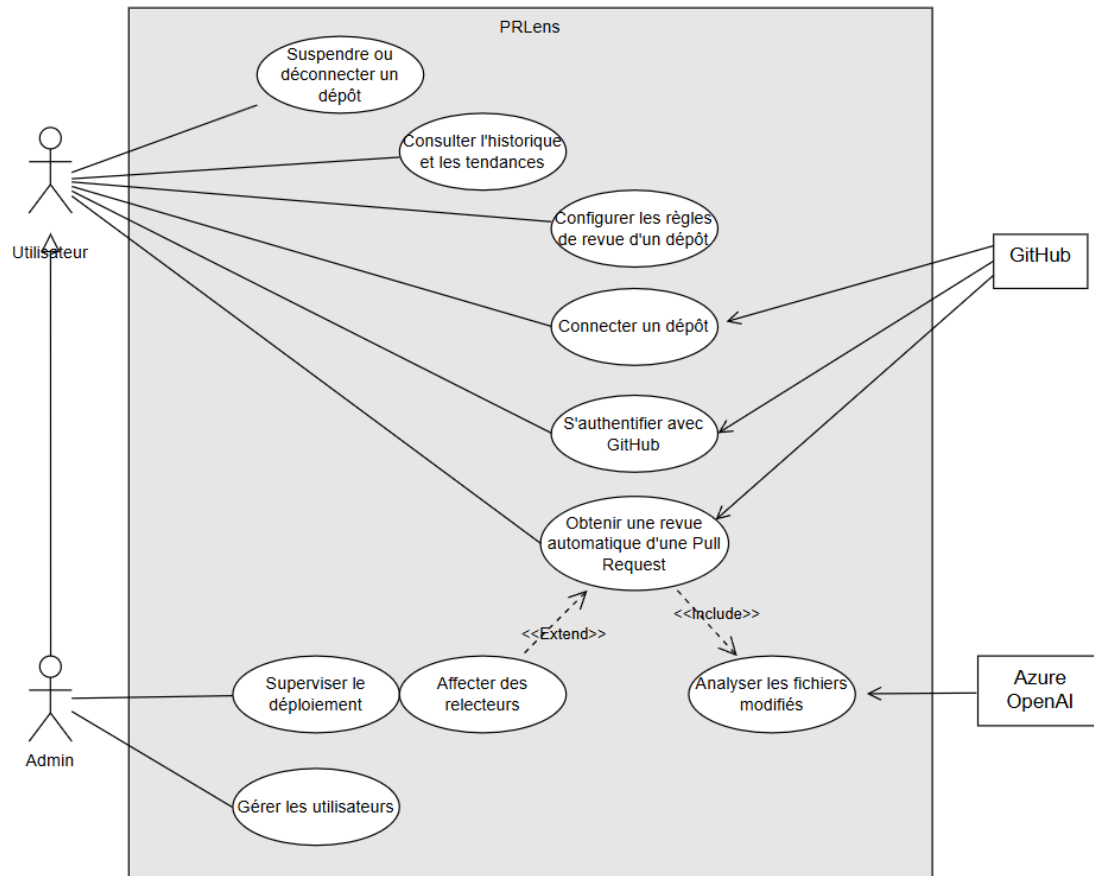


Figure 2.1 – Diagramme de cas d'utilisation global du système PRLens

2.5 Description textuelle des cas d'utilisation

Les trois cas d'utilisation les plus structurants font l'objet d'une description textuelle détaillée.

Cas d'utilisation : S'authentifier	
Acteur principal	Utilisateur
Précondition	L'utilisateur possède un compte GitHub actif.
Postcondition	L'utilisateur dispose d'un jeton de session valide et accède à son tableau de bord.
Scénario nominal	1. L'utilisateur clique sur « Se connecter avec GitHub ». 2. Le système le redirige vers la page d'autorisation GitHub. 3. L'utilisateur autorise l'application. 4. GitHub redirige vers le point de retour avec un code temporaire. 5. Le système échange ce code contre un jeton d'accès, récupère le profil, crée ou met à jour le compte, puis émet un jeton de session.
Scénario alternatif	Si le code est expiré ou déjà consommé, GitHub répond sans jeton d'accès ; le système signale l'échec et renvoie l'utilisateur vers la page de connexion.

Tableau 2.4 – Description textuelle du cas d'utilisation « S'authentifier »

Cas d'utilisation : Connecter un dépôt	
Acteur principal	Utilisateur
Précondition	L'utilisateur est authentifié et l'application GitHub est installée sur le dépôt visé.
Postcondition	Le dépôt est enregistré comme installation active ; ses Pull Requests seront désormais revues automatiquement.
Scénario nominal	1. L'utilisateur ouvre la page de connexion des dépôts. 2. Le système liste les dépôts GitHub accessibles à son compte. 3. L'utilisateur sélectionne un dépôt et confirme. 4. Le système crée l'installation avec les paramètres de revue par défaut.
Scénario alternatif	Si le dépôt est déjà connecté au même compte, la contrainte d'unicité sur le couple (utilisateur, dépôt) empêche la création d'un doublon et le système signale que le dépôt est déjà rattaché.

Tableau 2.5 – Description textuelle du cas d'utilisation « Connecter un dépôt »

Cas d'utilisation : Recevoir une revue automatique	
Acteur principal	Développeur
Acteurs secondaires	GitHub, Service LLM
Précondition	Le dépôt est connecté et actif.
Postcondition	La Pull Request porte les commentaires en ligne, le commentaire de synthèse, les étiquettes et le verdict ; la revue est enregistrée dans l'historique.
Scénario nominal	1. Le développeur ouvre ou met à jour une Pull Request. 2. GitHub émet un événement vers le point d'entrée du système. 3. Le système vérifie la signature de la requête et déclenche l'analyse en tâche de fond. 4. Chaque fichier retenu est soumis au service LLM, en parallèle. 5. Le système agrège les constats, calcule le score et le verdict. 6. Le système publie le résultat sur la Pull Request et l'enregistre en base.
Scénario alternatif	Si l'analyse d'une partie des fichiers échoue, le système publie néanmoins les constats obtenus et marque la revue comme incomplète. Si aucun fichier n'a pu être analysé, la revue est marquée en échec total.

Tableau 2.6 – Description textuelle du cas d'utilisation « Recevoir une revue automatique »

2.6 Diagrammes de séquence

2.6.1 Authentification et connexion d'un dépôt

La figure 2.2 illustre le parcours complet d'un utilisateur, de la connexion par OAuth jusqu'à la réception d'une revue automatique sur une Pull Request ouverte sur un dépôt qu'il vient de connecter.

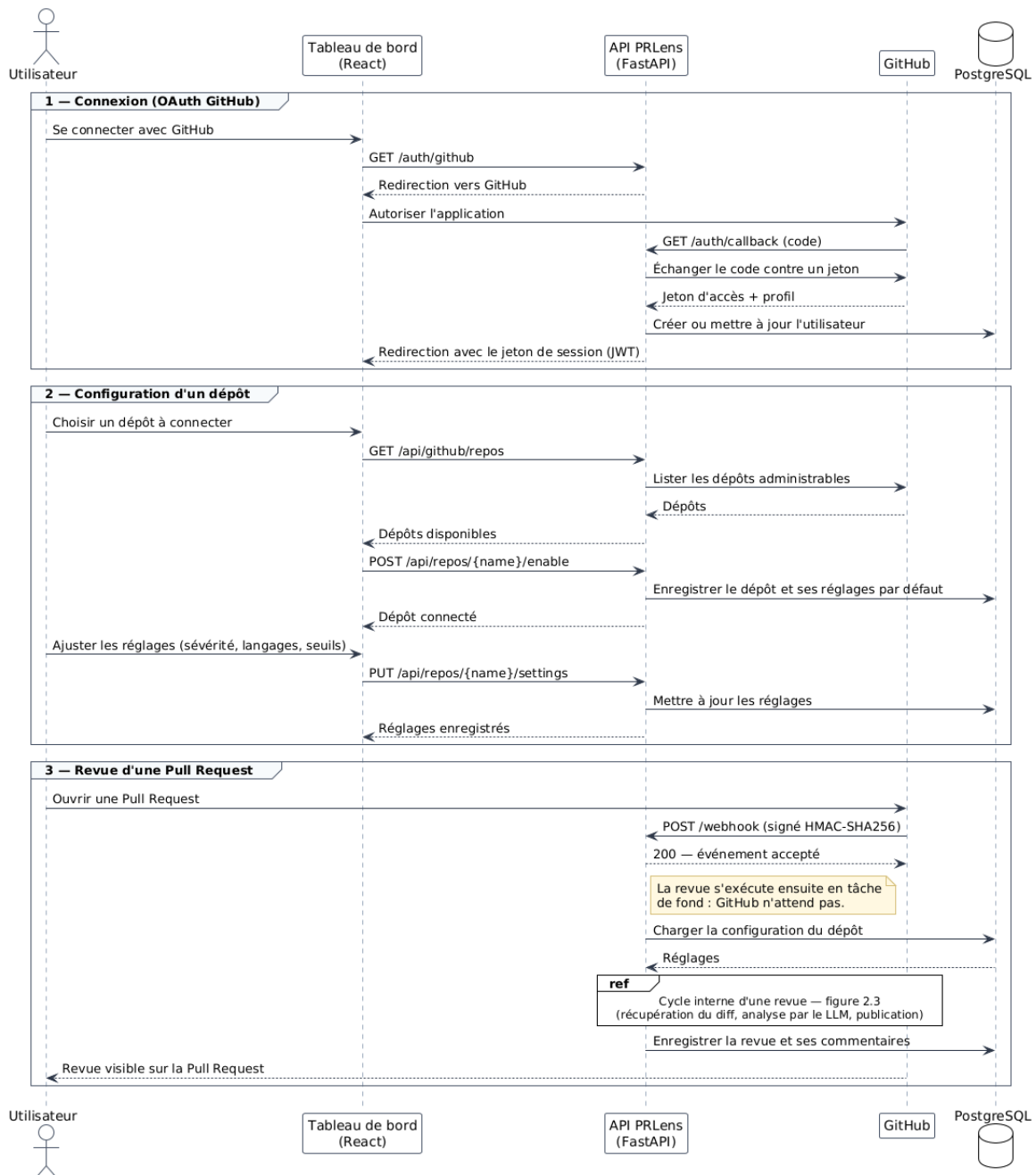


Figure 2.2 — Diagramme de séquence — connexion, configuration et revue d'une Pull Request

2.6.2 Cycle interne d'une revue

La figure 2.3 détaille le cycle interne de l'analyse, depuis la réception de l'événement jusqu'à la publication du résultat, en faisant apparaître la parallélisation des appels au service LLM.

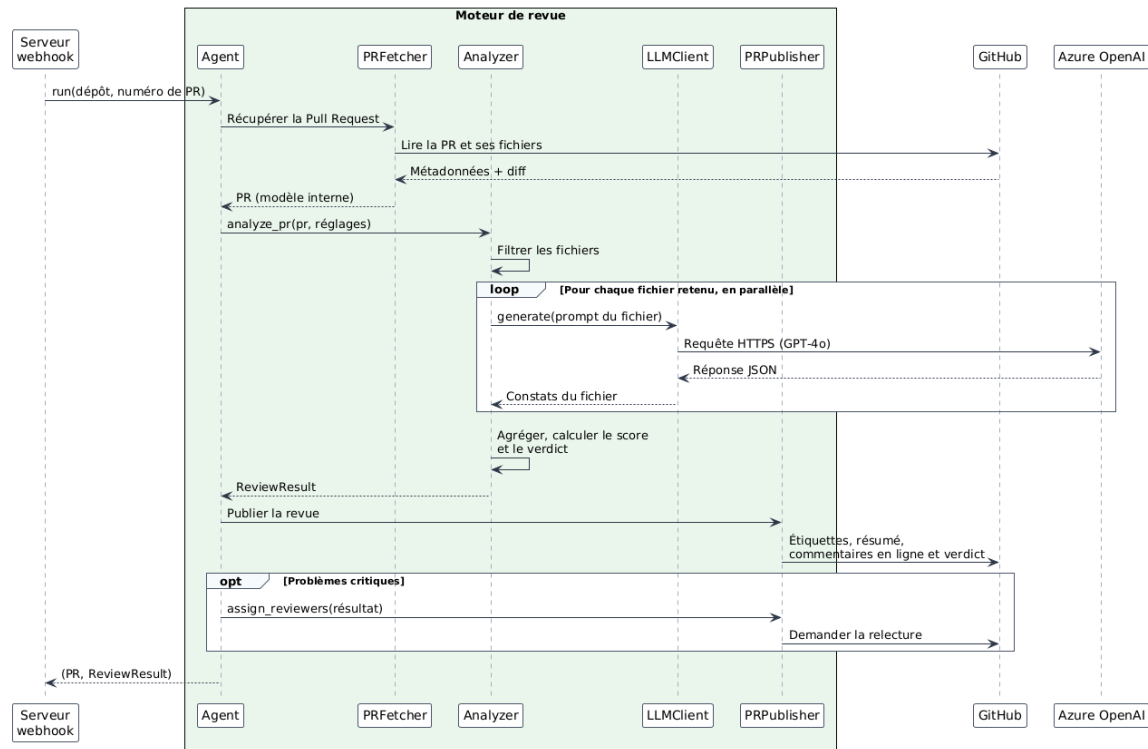


Figure 2.3 – Diagramme de séquence — cycle interne d'une revue

2.7 Backlog produit

Le backlog produit organise les besoins fonctionnels en éléments priorisés et rattachés à un module. Le tableau 2.7 en présente la synthèse.

ID	Priorité	Fonctionnalité	Module
BF01	Haute	Analyser un <i>diff</i> de Pull Request et produire des commentaires classés par sévérité	Moteur
BF03	Haute	Calculer un score de qualité et un verdict de revue formel	Moteur
—	Haute	Authentifier l'application auprès de GitHub (App ou jeton personnel)	Moteur
—	Haute	Exécuter la revue depuis un pipeline GitHub Actions	Moteur
—	Haute	Exécuter la revue depuis un serveur webhook permanent	Moteur
BF07	Moyenne	Permettre la connexion des utilisateurs par OAuth GitHub	Dashboard
BF08	Moyenne	Connecter et déconnecter un dépôt depuis l'interface	Dashboard
BF06	Moyenne	Configurer par dépôt les seuils, langages et fichiers exclus	Dashboard
BF10	Moyenne	Afficher l'historique des revues et les tendances de score	Dashboard
BF09	Basse	Suspendre temporairement la revue d'un dépôt	Dashboard
BF05	Basse	Affecter automatiquement des relecteurs selon le type de problème	Moteur
BF11	Basse	Offrir à l'exploitant une vue d'ensemble du déploiement	Dashboard

Tableau 2.7 – Backlog produit du projet PRLens

2.8 Planification des itérations

Le tableau 2.8 détaille le contenu de chaque itération et le livrable associé.

Itération	Contenu	Livrable
Itération 1	Modèles de données, client GitHub, analyseur, publication, orchestrateur	Moteur de revue fonctionnel en mode Actions
Itération 2	Serveur FastAPI, vérification de signature, tâches de fond, persistance	Mode webhook opérationnel
Itération 3	OAuth GitHub, API REST, interface React, connexion des dépôts	Tableau de bord utilisable
Itération 4	Statistiques, historique, espace d'administration, limite d'utilisation, déploiement	Service en production

Tableau 2.8 – Planification des itérations du projet

Conclusion

Ce chapitre a identifié les acteurs du système, spécifié ses besoins fonctionnels et non fonctionnels, et modélisé les interactions correspondantes. Ces spécifications constituent le cahier des charges auquel répond la conception détaillée présentée au chapitre suivant.

CHAPITRE 3

Conception

Sommaire du chapitre

1. Introduction
2. Architecture globale du système
3. Architecture technique
4. Diagramme de déploiement
5. Diagramme d'activité — le cycle de revue
6. Diagramme de classes
7. Conception de la base de données
8. Maquettes de l'interface
9. Algorithmes et règles de décision
10. Sécurité et gestion des accès
11. Choix technologiques et justifications
12. Conclusion

Introduction

Ce chapitre présente la conception détaillée de PRLens à partir des besoins spécifiés au chapitre précédent. L'objectif principal de cette phase a été de définir une architecture capable de servir un même pipeline de revue depuis deux points d'entrée distincts, tout en isolant proprement la couche de persistance nécessaire au tableau de bord.

3.1 Architecture globale du système

PRLens partage un unique pipeline de revue — la classe **Agent** — derrière deux points d'entrée. En mode *Actions*, le script `main.py` l'exécute une fois par événement de Pull Request, à l'intérieur d'un conteneur Docker. En mode *webhook*, un serveur FastAPI invoque ce même pipeline à chaque événement reçu, et persiste en plus le résultat pour le tableau de bord. La figure 3.1 illustre ce flux.

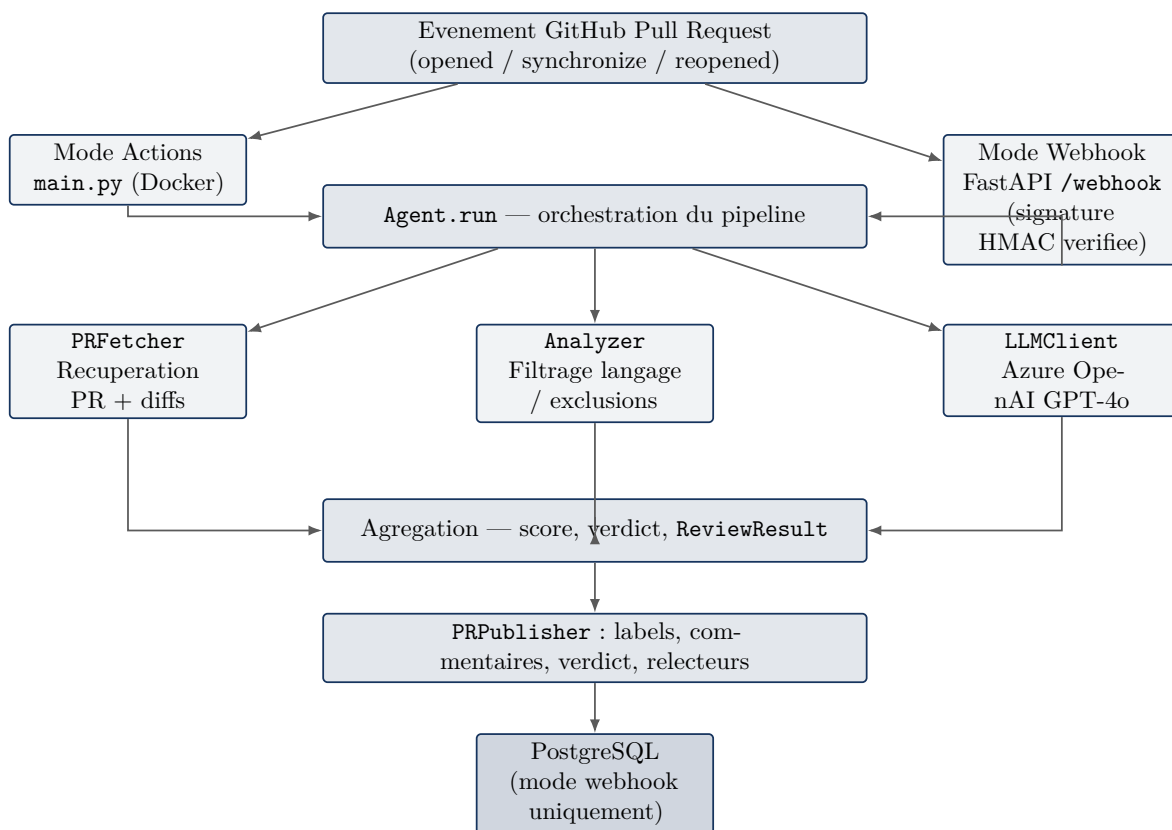


Figure 3.1 – Architecture globale du pipeline de revue PRLens

3.2 Architecture technique

L'application est organisée en couches dont les dépendances pointent toutes dans le même sens : les points d'entrée connaissent l'orchestrateur, l'orchestrateur connaît

les adaptateurs, et les adaptateurs connaissent le modèle du domaine. Aucun cycle de dépendance n'existe entre les paquets.

3.2.1 Le moteur de revue

Le moteur est écrit en Python et réparti en quatre paquets :

- **config** — chargement du fichier `.aireviewer.yml` et modèle `Settings`, qui porte l'ensemble des paramètres réglables (langages, exclusions, seuils, nombre de travailleurs).
- **github** — `GitHubClient` gère l'authentification (jeton personnel ou application GitHub) et le renouvellement des jetons ; `PRFetcher` récupère la Pull Request et la convertit en modèles internes ; `PRPublisher` traduit le résultat en actions GitHub.
- **llm** — `LLMClient` encapsule l'appel à l'API et impose un contrat de sortie JSON strict ; `Analyzer` filtre les fichiers, distribue les appels et agrège les constats.
- **models** — les entités du domaine (`PR`, `FileChange`, `ReviewResult`, `ReviewComment`), validées par Pydantic.

La classe `Agent` du paquet `core` orchestre ces composants sans effectuer elle-même la moindre entrée-sortie : c'est ce qui la rend testable avec de simples doublures.

3.2.2 Le serveur webhook et l'API REST

Le serveur FastAPI assure trois rôles distincts : réception et vérification des événements GitHub, exposition de l'API REST consommée par le tableau de bord, et persistance des revues effectuées. L'analyse elle-même est déclenchée en tâche de fond, afin de répondre immédiatement à GitHub, qui impose un délai de réponse court.

3.2.3 Le tableau de bord

Le tableau de bord est une application web monopage écrite en TypeScript avec React, structurée en pages (`src/pages`), composants réutilisables (`src/components`) et une couche d'accès à l'API typée (`src/lib/api.ts`) qui centralise la gestion du jeton d'authentification.

3.3 Diagramme de déploiement

La figure 3.2 présente la répartition des composants sur les nœuds d'exécution en production.

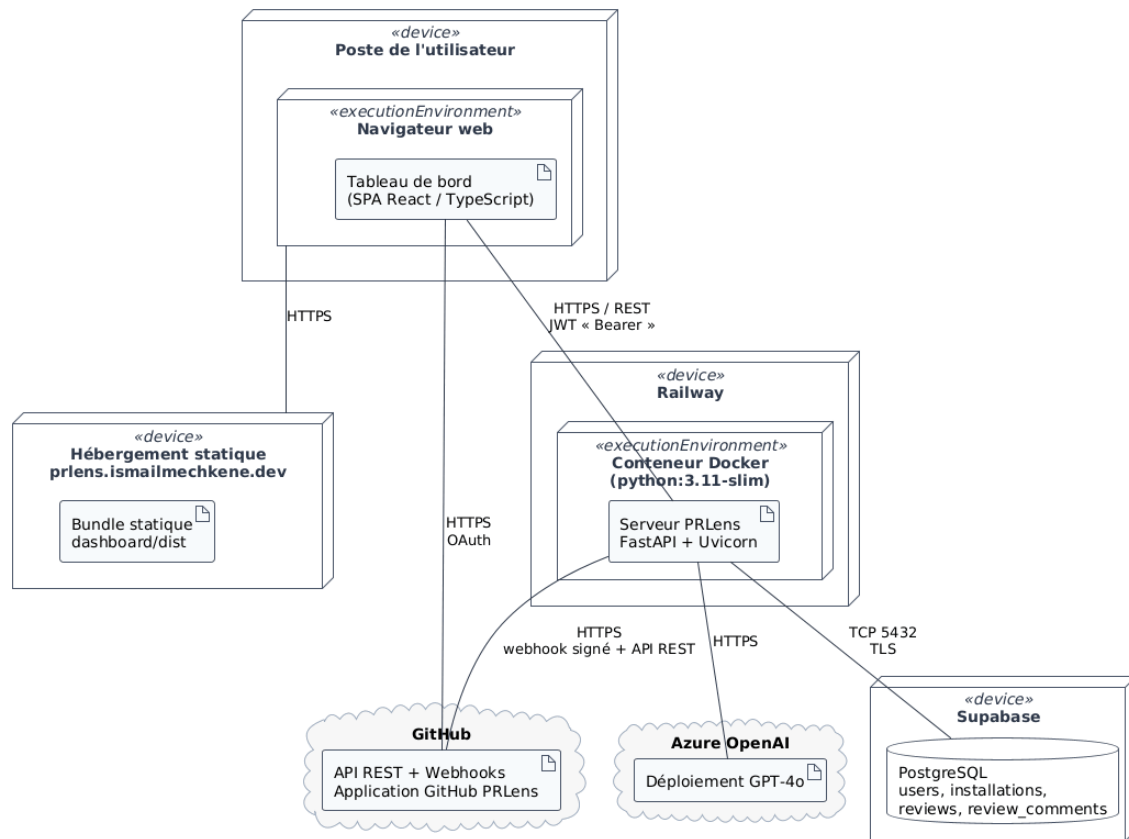


Figure 3.2 – Diagramme de déploiement de PRLens en production

3.4 Diagramme d'activité — le cycle de revue

La figure 3.3 décrit le cheminement complet d'une revue, en faisant apparaître le point de parallélisation (un appel au modèle par fichier retenu) et les points de décision qui déterminent le verdict final.

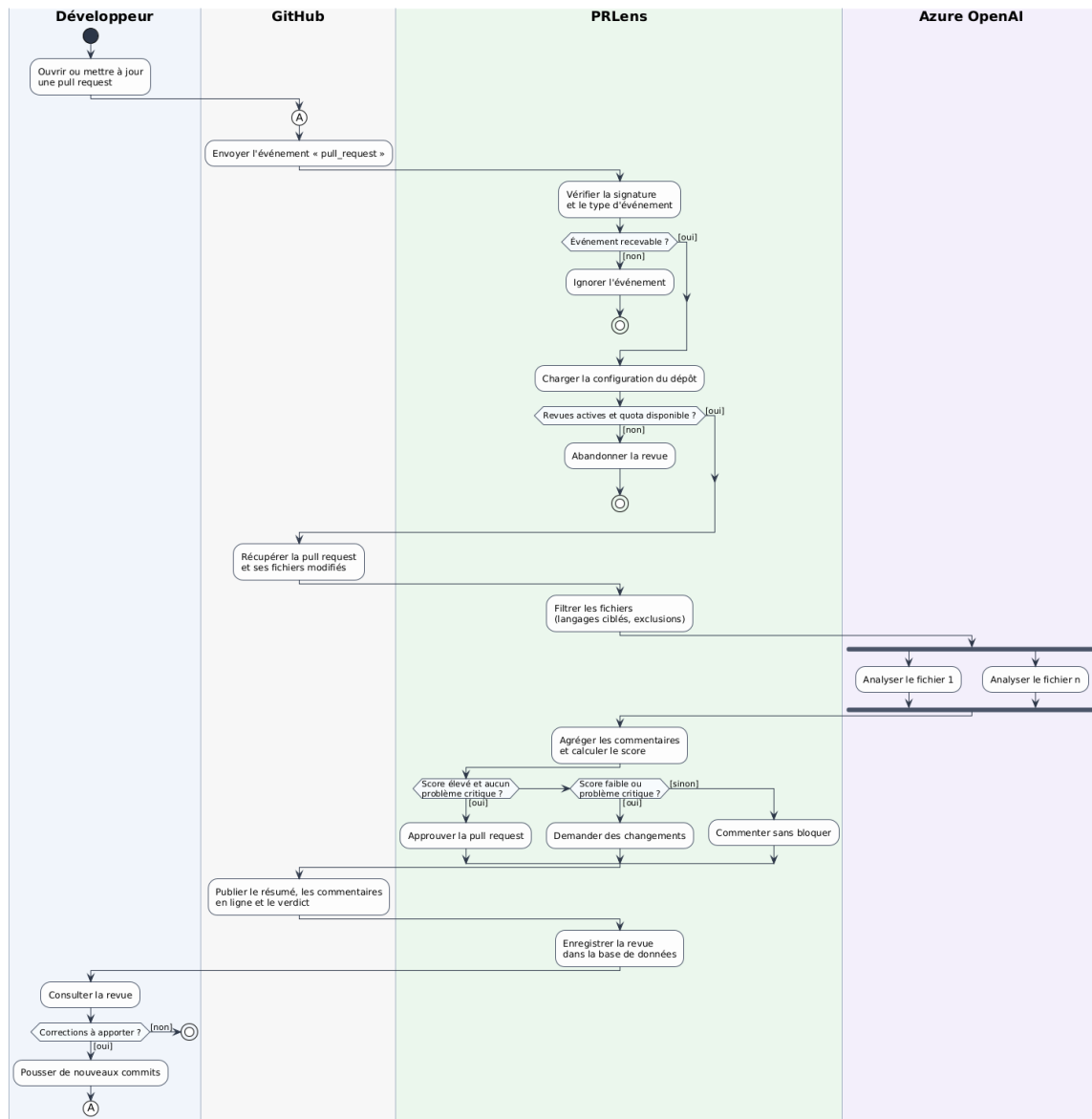


Figure 3.3 – Diagramme d'activité du cycle de revue

3.5 Diagramme de classes

La figure 3.4 présente le diagramme de classes du moteur de revue. Les collaborateurs sont injectés par constructeur, ce qui se traduit par des relations d'agrégation ; les entités du domaine et leurs constituants, dont le cycle de vie est lié, sont reliés par des relations de composition.

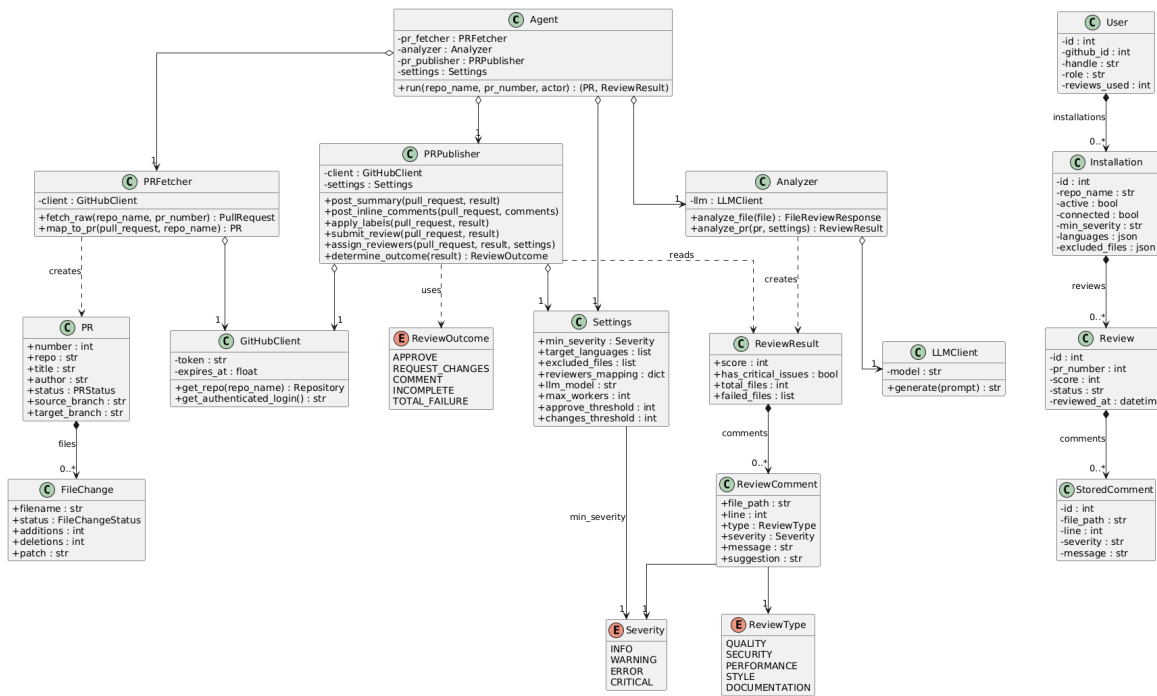


Figure 3.4 – Diagramme de classes du moteur de revue

3.6 Conception de la base de données

La persistance nécessaire au tableau de bord repose sur quatre entités principales : **User**, **Installation**, **Review** et **ReviewComment**. La figure 3.5 en présente le schéma relationnel.

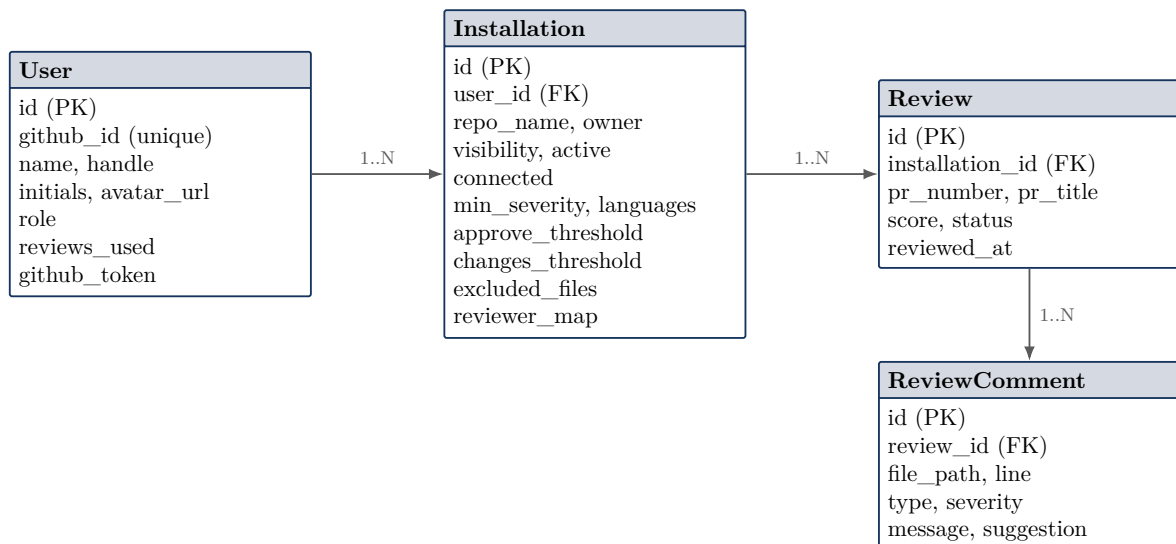


Figure 3.5 – Modèle de données relationnel (extrait des modèles SQLAlchemy)

Un utilisateur possède plusieurs installations, chacune représentant le rattachement d'un dépôt GitHub à un compte, avec ses paramètres de revue propres. Chaque installation accumule un historique de revues, et chaque revue détaille ses constats

individuels. La contrainte d'unicité portant sur le couple (`user_id`, `repo_name`) garantit qu'un même utilisateur ne connecte pas deux fois le même dépôt.

Les relations sont déclarées avec `cascade="all, delete-orphan"` : la suppression d'un utilisateur entraîne celle de ses installations, et la déconnexion d'une installation entraîne celle de son historique de revues, ce qui garantit l'absence d'enregistrements orphelins.

L'attribut `role` de `User` prend l'une de deux valeurs, `user` ou `admin`, et constitue le seul mécanisme d'autorisation du système. Un administrateur reste par ailleurs un utilisateur ordinaire — il possède ses propres installations et son propre tableau de bord ; le rôle *ajoute* des droits de consultation, il n'en retire aucun.

3.7 Maquettes de l'interface

Les maquettes retenues pour le tableau de bord se répartissent en trois écrans : la page d'accueil publique (figure 3.6), qui présente le service et porte le point d'entrée de l'authentification ; le tableau de bord proprement dit (figure 3.7), qui agrège les indicateurs de tous les dépôts connectés ; et la page de détail d'un dépôt (figure 3.8), qui regroupe ses paramètres de revue et son historique.

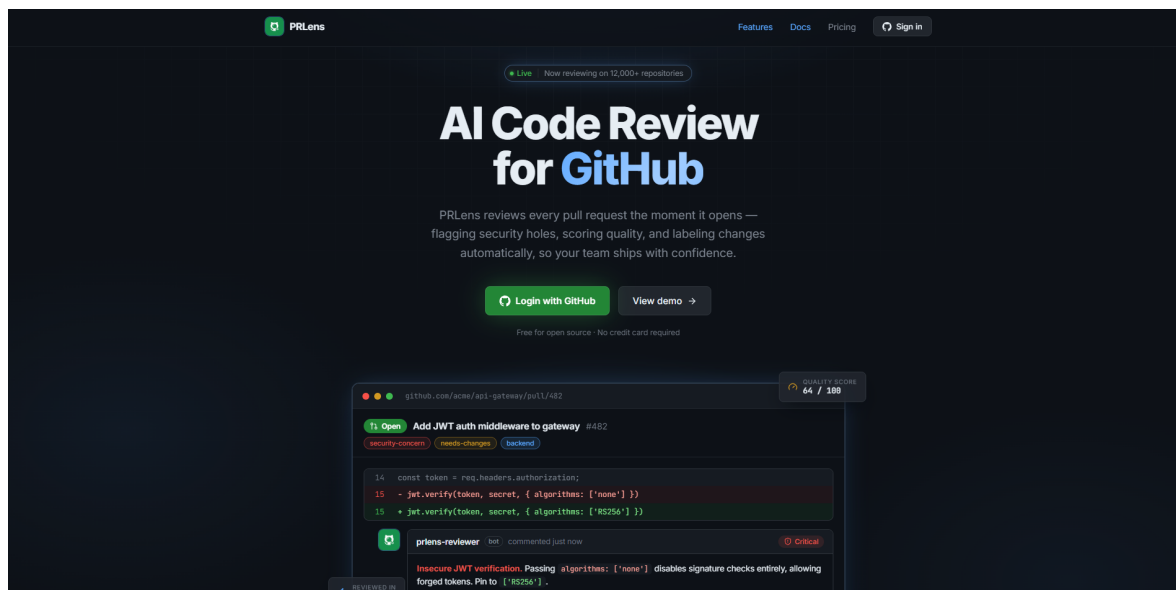


Figure 3.6 – Maquette de la page d'accueil publique

L'authentification s'appuie exclusivement sur GitHub, évitant à l'utilisateur la création d'un compte et d'un mot de passe supplémentaires. La bascule *actif* / *inactif* par dépôt permet de suspendre les revues sans perdre l'historique déjà accumulé, un choix ergonomique motivé par le besoin d'essais temporaires, par exemple pendant une migration de code volumineuse.

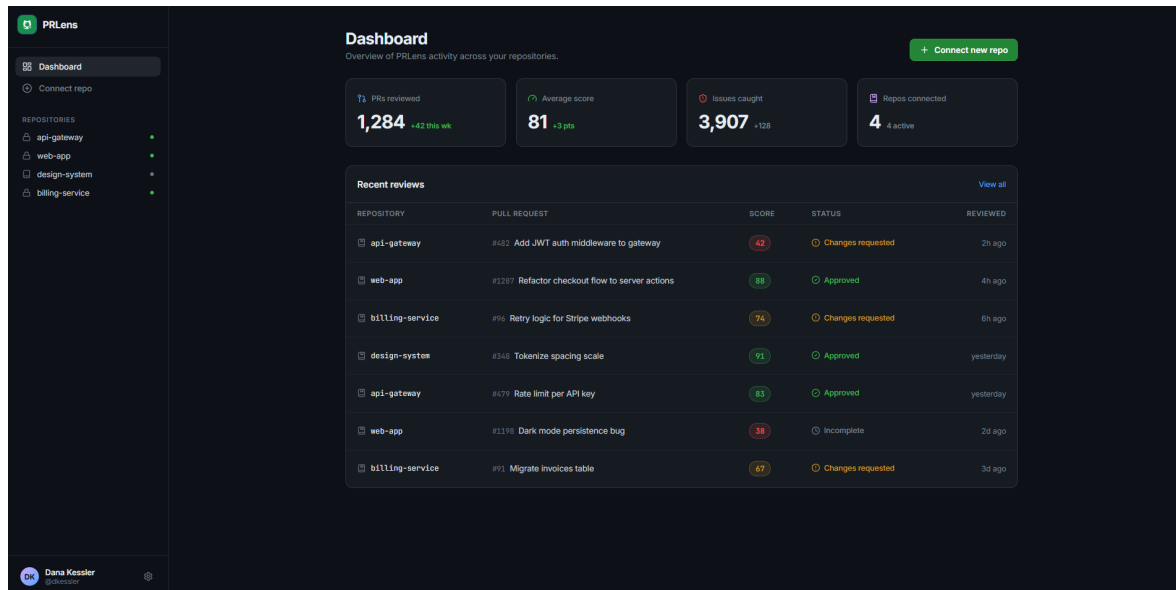


Figure 3.7 – Maquette de l'interface principale du tableau de bord

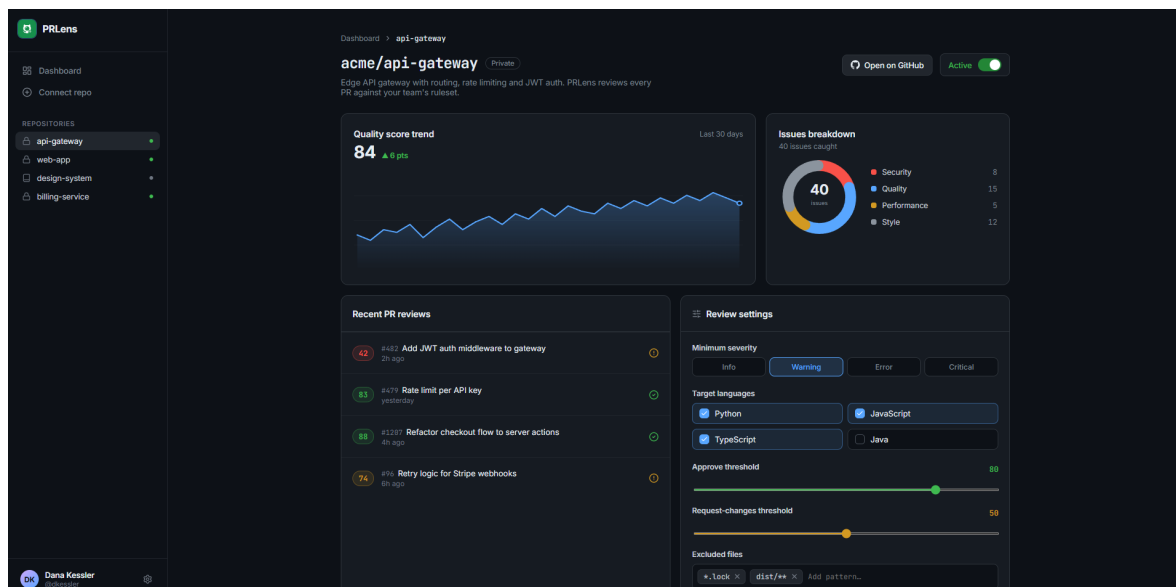


Figure 3.8 – Maquette de la page de détail d'un dépôt

3.8 Algorithmes et règles de décision

3.8.1 Analyse parallèle des fichiers

Pour chaque fichier retenu par le filtre de langage et d'exclusion, l'analyseur soumet un appel indépendant au modèle. Ces appels sont distribués sur un *pool* de travailleurs dont la taille est bornée par le paramètre `max_workers` (5 par défaut). Cette borne remplit deux fonctions : elle parallélise l'analyse d'une Pull Request touchant plusieurs fichiers, et elle limite le nombre de requêtes simultanées envoyées au fournisseur de LLM, dont le débit est contingenté.

3.8.2 Calcul du score et du verdict

Le score de qualité est calculé sur une échelle de 0 à 100, par pénalités cumulées selon la sévérité de chaque constat remonté : -25 points par problème critique, -10 par erreur, -4 par avertissement et -1 par information. Le verdict final est ensuite dérivé de ce score et de la présence ou non de problèmes critiques, selon la logique présentée au tableau 3.1.

Condition	Verdict	Paramètre
Score strictement supérieur au seuil ET aucun problème critique	Approve	<code>approve_threshold</code> (80 par défaut)
Score inférieur au seuil OU au moins un problème critique	Request changes	<code>changes_threshold</code> (50 par défaut)
Cas intermédiaire	Comment	—

Tableau 3.1 – Règles de décision du verdict de revue

Cette séparation est essentielle : le modèle de langage produit des *constats*, mais il ne décide de rien. Le score et le verdict sont calculés par du code déterministe à partir de ces constats, ce qui garantit qu'à constats identiques le résultat publié est identique.

3.8.3 Gestion des échecs partiels

Lorsque l'analyse d'un ou de plusieurs fichiers échoue — erreur réseau, réponse non conforme au schéma attendu — PRLens distingue un état *incomplet*, où une partie des fichiers a pu être analysée, d'un état d'*échec total*, où aucun fichier n'a pu l'être. Cette distinction évite de faire échouer silencieusement l'ensemble de la revue et permet de faire remonter l'incident dans les statistiques d'exploitation.

De la même manière, une réponse du modèle contenant un constat malformé n'invalide pas les autres : les constats sont validés un à un et seuls les irréparables sont écartés.

3.9 Sécurité et gestion des accès

3.9.1 Mesures de sécurité

- **Vérification de signature webhook** — chaque requête reçue est vérifiée par comparaison HMAC-SHA256 de l'en-tête `X-Hub-Signature-256` avec le secret partagé ; toute requête non conforme est rejetée.

- **Jetons à durée de vie courte** — l’authentification en tant qu’application GitHub repose sur un JWT signé avec la clé privée de l’application, échangé contre un jeton d’installation renouvelé automatiquement avant expiration.
- **Session sans cookie inter-domaine** — l’API et le tableau de bord pouvant être servis sur des domaines distincts, l’authentification utilisateur repose sur un jeton de session transmis en en-tête **Authorization: Bearer**, et non sur un cookie.
- **Clé de signature obligatoire en production** — en l’absence de `SESSION_SECRET`, une clé aléatoire par processus est utilisée et un avertissement est journalisé au démarrage.
- **Protection des secrets** — la clé privée de l’application GitHub est manipulée sous forme encodée en base64 dans les environnements conteneurisés, les fichiers `.pem` étant exclus du dépôt et de l’image Docker. Les jetons GitHub des utilisateurs sont conservés côté serveur uniquement.

3.9.2 Espace d’administration et moindre privilège

Toutes les vues du tableau de bord sont *cloisonnées par utilisateur* : elles ne renvoient que les données rattachées au compte appelant. Cette propriété est nécessaire, mais elle a une conséquence structurelle : personne ne dispose d’une vue de l’état global du déploiement. Or certains signaux ne sont observables qu’à cette échelle. Une remontée du taux d’échec des revues sur des dépôts *sans lien entre eux* et appartenant à des utilisateurs différents ne traduit pas une dégradation du code relu, mais une défaillance du service LLM sous-jacent ; aucun tableau de bord individuel ne peut, par construction, faire apparaître cette corrélation.

L’espace d’administration répond à ce besoin. Deux propriétés de conception méritent d’être soulignées :

- **Le rôle est relu en base à chaque requête**, et non porté comme revendication dans le jeton de session. Un jeton émis alors que l’utilisateur était administrateur cesse donc d’ouvrir ces routes dès la révocation du rôle, et non à l’expiration du jeton — soit jusqu’à trente jours plus tard.
- **Le masquage du lien dans l’interface n’est pas le contrôle d’accès**. La barre latérale n’affiche l’entrée *Admin* qu’aux administrateurs, et une saisie manuelle de l’URL redirige l’utilisateur ordinaire vers son propre tableau de bord ; ces deux mécanismes sont purement ergonomiques. La vérification qui fait autorité est celle du serveur.

L’espace est par ailleurs **en lecture seule** : il rend compte de l’état des dépôts des autres utilisateurs, mais ne permet ni de les reconfigurer, ni de les déconnecter, ni de promouvoir un compte. Ce choix est délibérément restrictif : une interface

d'administration capable d'écrire — et a fortiori capable d'accorder le rôle d'administrateur — transformerait la compromission d'une seule session privilégiée en une escalade de privilèges durable. L'attribution du rôle se fait donc hors bande, par un script d'administration exécuté par une personne disposant déjà d'un accès à la base.

3.10 Choix technologiques et justifications

Le tableau 3.2 récapitule la pile technologique retenue et la justification de chaque choix.

Couche	Technologie	Justification
Langage — moteur	Python 3.11	Écosystème mature pour l'intégration d'API et le traitement de données
Framework serveur	FastAPI	Validation native par Pydantic (mêmes modèles que le moteur) et tâches de fond sans bloquer la réponse HTTP attendue par GitHub
Client GitHub	PyGithub	Couverture complète de l'API REST de GitHub
Client LLM	SDK OpenAI (Azure OpenAI)	Mode de sortie JSON strict, indispensable pour exploiter la réponse programmatiquement
Validation	Pydantic v2	Modélisation stricte des entités et récupération gracieuse des réponses partiellement malformées
Persistance	PostgreSQL + SQLAlchemy	Maturité du SGBD et support natif du type JSON pour les paramètres à structure variable
Serveur ASGI	Uvicorn	Exécution performante du serveur FastAPI
Langage — interface	TypeScript	Typage statique sur des données structurées issues de l'API
Framework interface	React 19 + Vite	Écosystème riche et temps de compilation courts
Authentification	OAuth 2.0 + JWT	Aucun mot de passe à stocker ; identité déléguée à GitHub
Conteneurisation	Docker	Empaquetage reproductible de l'action GitHub Actions

Tableau 3.2 – Pile technologique du projet et justification des choix

Conclusion

Ce chapitre a détaillé la conception retenue : une architecture partagée entre deux points d'entrée, un modèle de données relationnel simple mais suffisant, une logique de décision entièrement déterministe construite au-dessus d'une source qui ne l'est pas, et des mécanismes de sécurité adaptés à un système dialoguant avec des services tiers sensibles. Le chapitre suivant décrit la réalisation effective de cette conception et sa validation.

CHAPITRE 4

Réalisation

Sommaire du chapitre

1. Introduction
2. Environnement de développement
3. Itération 1 — Moteur de revue et authentification GitHub (semaines 1–2)
4. Itération 2 — Mode webhook et persistance (semaines 3–4)
5. Itération 3 — Tableau de bord et authentification OAuth (semaines 5–6)
6. Itération 4 — Historique, administration et quotas (semaines 7–9)
7. Tests et validation
 - Stratégie de tests
 - Résultats des tests unitaires
 - Harnais d'évaluation
8. Déploiement en production
9. Conclusion

Introduction

Ce chapitre présente la mise en œuvre concrète de la conception décrite au chapitre précédent, itération par itération, puis le protocole de validation retenu. Cette validation repose sur deux piliers complémentaires : une suite de tests unitaires portant sur la logique déterministe du système, et un harnais d’évaluation dédié mesurant la pertinence des revues produites par le modèle de langage, dont le comportement est par nature non déterministe.

4.1 Environnement de développement

Catégorie	Outils et technologies
Langages	Python 3.11, TypeScript
Environnement de développement	PyCharm, Visual Studio Code
Gestion de versions	Git, GitHub
Backend	FastAPI, Uvicorn, SQLAlchemy, Pydantic v2
Frontend	React 19, Vite, React Router
Base de données	PostgreSQL (Supabase en production, SQLite en test)
Service LLM	Azure OpenAI (déploiement <code>gpt-4o</code>)
Tests	Pytest, <code>pytest-cov</code>
Qualité de code	Ruff (analyse statique), <code>tsc</code> et ESLint côté interface
Conteneurisation	Docker
Hébergement	Railway (serveur), Supabase (base de données)

Tableau 4.1 – Environnement de développement et technologies utilisées

4.2 Itération 1 — Moteur de revue et authentification GitHub (semaines 1–2)

4.2.1 Objectifs

Cette première itération visait à produire un moteur de revue fonctionnel de bout en bout, exécutable depuis un pipeline GitHub Actions : récupérer une Pull Request, analyser ses fichiers modifiés et publier le résultat, sans aucune infrastructure serveur.

4.2.2 Réalisation

Les modèles du domaine ont été définis en premier, car ils constituent le contrat partagé par tous les autres composants. Le client GitHub a ensuite été implémenté avec deux stratégies d'authentification interchangeables : un jeton personnel, suffisant pour les essais, et une application GitHub, nécessaire pour un usage sur plusieurs dépôts. Cette dernière repose sur un JWT signé avec la clé privée de l'application, échangé contre un jeton d'installation à durée de vie courte, renouvelé automatiquement.

L'analyseur soumet ensuite chaque fichier retenu au modèle de langage, avec une instruction système imposant un format de réponse JSON strict. Le publieur traduit enfin le résultat agrégé en actions GitHub : étiquettes, commentaire de synthèse, commentaires en ligne, verdict formel et affectation de relecteurs.

4.2.3 Résultats

À l'issue de cette itération, PRLens était capable de revoir automatiquement ses propres Pull Requests. La figure 4.1 montre un exemple de commentaire publié sur une ligne du *diff*.

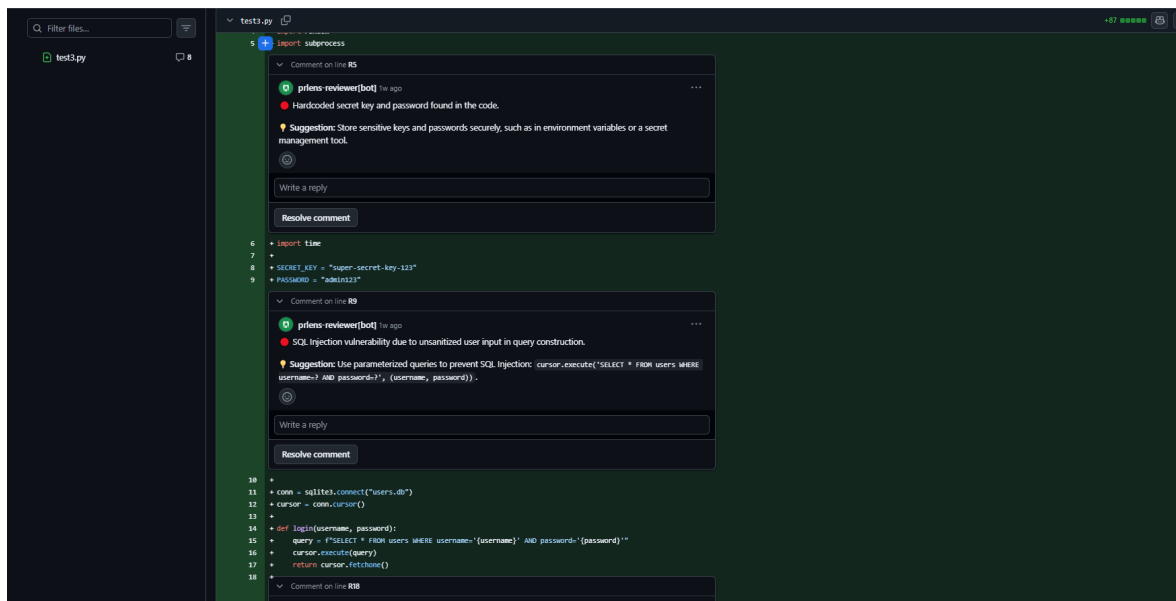


Figure 4.1 – Commentaire en ligne généré automatiquement sur une Pull Request

4.3 Itération 2 — Mode webhook et persistance (semaines 3–4)

4.3.1 Objectifs

Offrir un second mode de déploiement, dans lequel l’application est installée une seule fois pour tous les dépôts d’une organisation, sans exiger l’ajout d’un fichier de workflow dans chacun d’eux — et enregistrer chaque revue en base pour permettre, à terme, l’affichage d’un historique.

4.3.2 Réalisation

Un serveur FastAPI expose un point d’entrée qui vérifie la signature HMAC de chaque livraison avant tout traitement. L’analyse étant trop longue pour tenir dans le délai de réponse attendu par GitHub, elle est déclenchée en tâche de fond et la réponse est renvoyée immédiatement. Un mécanisme d’anti-rebond évite par ailleurs qu’une série rapide de *commits* ne déclenche autant de revues redondantes.

Le schéma relationnel décrit au chapitre précédent a été implémenté avec SQLAlchemy, chaque revue terminée étant enregistrée avec son score, son statut et l’ensemble de ses constats.

4.3.3 Résultats

Les deux modes de déploiement partagent le même code d’orchestration, ce qui garantit qu’ils produisent un résultat identique pour un même contenu de Pull Request.

4.4 Itération 3 — Tableau de bord et authentification OAuth (semaines 5–6)

4.4.1 Objectifs

Permettre à un utilisateur de gérer l’ensemble du système depuis une interface web, sans jamais éditer de fichier de configuration ni intervenir en base.

4.4.2 Réalisation

L’authentification repose sur le flux OAuth 2.0 de GitHub : le serveur échange le code temporaire reçu contre un jeton d’accès, récupère le profil de l’utilisateur, crée ou met à jour son compte et émet un jeton de session signé. L’identifiant numérique GitHub, et non le pseudonyme, sert de clé d’identité — un pseudonyme peut être changé, puis réenregistré par une autre personne.

L’interface React consomme une API REST typée, et la page de connexion des dépôts liste les dépôts GitHub accessibles au compte pour permettre leur rattachement en un clic.

4.5 Itération 4 — Historique, administration et quotas (semaines 7–9)

4.5.1 Objectifs

Compléter le tableau de bord par les vues d’analyse (statistiques, tendances, répartition des problèmes), doter l’exploitant d’une vue globale du déploiement, et borner la consommation de ressources par les comptes non administrateurs.

4.5.2 Réalisation

Les vues statistiques agrègent l’historique des revues par dépôt et dans le temps. L’espace d’administration décrit au §3.9.2 a été ajouté avec sa règle d’autorisation vérifiée côté serveur. Enfin, une limite d’utilisation plafonne le nombre de revues des comptes ordinaires, décomptées par un compteur dédié plutôt que par un simple dénombrement des revues existantes — ce dernier serait remis à zéro par la déconnexion d’un dépôt, dont les revues sont supprimées en cascade.

L’ajout des colonnes `role` et `reviews_used` sur une base déjà déployée a nécessité une attention particulière : la création automatique des tables ne modifie jamais une table existante. Ces colonnes sont donc ajoutées au démarrage par des instructions `ALTER TABLE` idempotentes, dont les valeurs par défaut rétro-remplissent les comptes existants.

4.5.3 Résultats

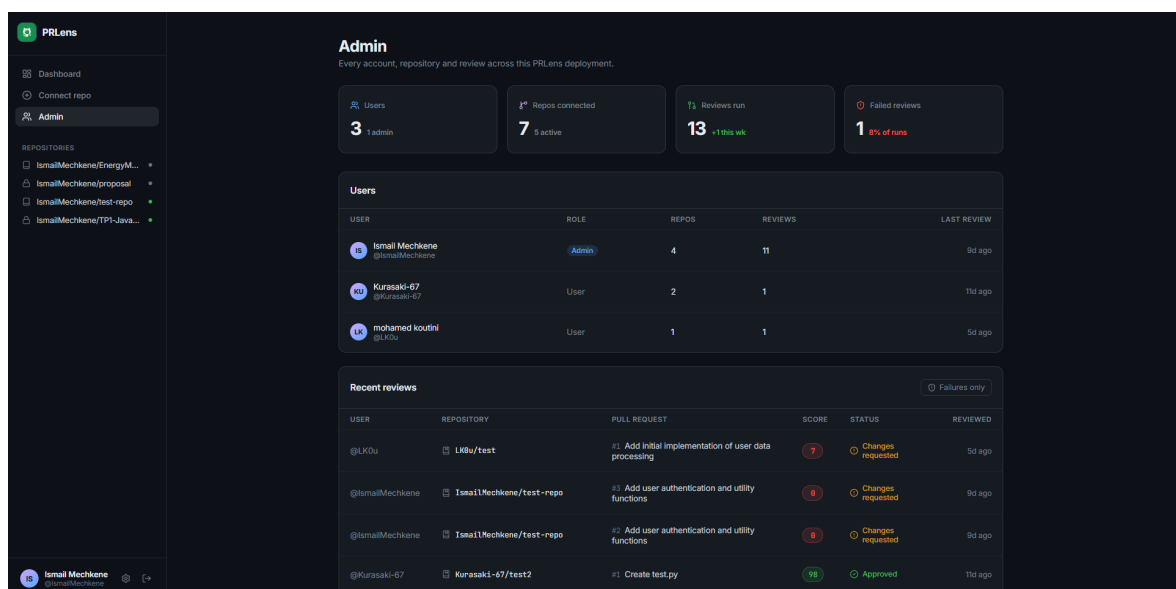


Figure 4.2 — Espace d’administration

4.6 Tests et validation

4.6.1 Stratégie de tests

La suite de tests unitaires s'exécute intégralement hors ligne : tous les appels à GitHub et au fournisseur de LLM sont simulés, ce qui garantit des tests rapides, déterministes et ne consommant aucun quota d'API externe. Elle couvre la configuration, le client GitHub (authentification par jeton personnel *et* par application), la récupération et la publication de Pull Requests, l'analyseur, le client LLM, l'orchestrateur et le serveur webhook.

Les tests de l'espace d'administration font exception à ce principe de simulation, et pour une raison de fond : la règle à vérifier porte précisément sur le rôle *stocké en base* ; simuler l'utilisateur reviendrait à ne rien tester du mécanisme d'autorisation. Ces tests s'exécutent donc contre une base SQLite en mémoire, avec des jetons de session réellement signés, de sorte que le chemin complet « jeton, puis utilisateur, puis rôle, puis décision » soit exercé de bout en bout. Chaque route d'administration est vérifiée sous trois angles : appelant anonyme (401), utilisateur authentifié mais ordinaire (403), et administrateur (200).

4.6.2 Résultats des tests unitaires

Au moment de la rédaction de ce rapport, la suite compte **147 tests, tous passants**. Le tableau 4.2 détaille la couverture de code par module.

Le moteur de revue — la partie déterministe et critique du système, responsable du calcul du score et du verdict — atteint une couverture de 93 %. Les modules de calcul proprement dits (analyseur, modèles, configuration, orchestrateur) sont couverts à 100 %. Le déficit se concentre sur `github/client.py`, dont la logique d'authentification par application GitHub a été enrichie en fin de projet par la résolution de l'identifiant d'installation dépôt par dépôt, sans que les tests correspondants aient suivi au même rythme.

La couche `webhook/app.py`, qui a fortement grossi avec l'ajout du tableau de bord, reste partiellement couverte : la vérification de signature, l'orchestration de la revue, la limite d'utilisation et l'intégralité de l'espace d'administration sont testées, mais la majorité des points d'accès cloisonnés par utilisateur ne le sont pas encore. Ces deux points constituent l'axe d'amélioration prioritaire identifié en conclusion.

4.6.3 Harnais d'évaluation

Les tests unitaires valident la logique déterministe du système, mais ils ne disent rien de la *pertinence* des revues produites — laquelle dépend du modèle de langage. Un **harnais d'évaluation** dédié a donc été développé : 35 *diffs* construits à la main,

Module	Instructions	Couverture
config/settings.py	42	100 %
core/agent.py	24	100 %
github/client.py	181	77 %
github/pr_fetcher.py	11	100 %
github/pr_publisher.py	178	98 %
llm/analyzer.py	58	100 %
llm/client.py	18	100 %
llm/prompts.py	8	100 %
models/pr.py	30	100 %
models/review.py	68	100 %
Moteur de revue (total)	618	93 %
webhook/app.py (webhook + API du tableau de bord)	543	59 %
Ensemble du projet	1161	77 %

Tableau 4.2 – Couverture de tests par module

chacun semant un problème précis dont le résultat attendu est connu, ou au contraire entièrement sain pour les cas dits « propres ».

Un cas n’est considéré réussi que si le système retourne un constat qui est **(a)** du type attendu, **(b)** à un niveau de sévérité au moins égal à celui attendu, et **(c)** effectivement relatif au problème semé — le message doit le mentionner. Cette troisième condition est déterminante : sans elle, un cas est validé par n’importe quel commentaire de forme à peu près correcte, et comme le type *qualité* est à la fois le plus fréquent et la valeur de repli d’un libellé non reconnu, la condition « un constat de qualité au moins informatif » est satisfaite par une remarque quelconque, y compris portant sur un tout autre sujet.

Le relecteur étant un modèle de langage, une exécution unique ne signifie rien : trois exécutions consécutives ont donné 88,6 %, 85,7 % et 82,9 %. C’est la **moyenne de 85,7 %** qui doit être retenue, et non la meilleure exécution — l’écart d’environ trois points est suffisamment large pour que deux mesures ponctuelles voisines soient indiscernables du bruit. Ce résultat dépasse largement le seuil de précision minimal de 70 % fixé pour le projet.

La **sécurité est la catégorie la plus solide**, avec 10 cas sur 10 réussis à chaque exécution mesurée. Trois cas échouent en revanche de manière constante, et il vaut la peine d’être précis sur les raisons, car deux d’entre eux relèvent moins d’une défaillance du relecteur que d’un désaccord entre l’évaluation et un choix de conception délibéré :

Catégorie	Cas	Réussis	Ce qui est vérifié
Sécurité	10	10	Secrets codés en dur, injection SQL, traversée de répertoire, injection de commande, désérialisation non sûre, hachage faible, SSRF, vérification TLS désactivée, XSS
Qualité	8	8	Imbrication excessive, duplication, except nu, fonctions démesurées, arguments par défaut mutables, nombres magiques, code mort
Performance	2	2	Concaténation quadratique, chargement intégral d'un fichier en mémoire
Style	2	1	Conventions de nommage incohérentes, noms de variables non descriptifs
Documentation	2	1	Absence de docstring sur une API publique, commentaire désynchronisé du code
Propre	6	5	Absence de faux positif sur du code effectivement correct
Mixte	5	5	Plusieurs types de problèmes dans un même <i>diff</i>
Total	35	31	88,6 % (meilleure exécution)

Tableau 4.3 – Résultats du harnais d'évaluation

- un faux positif authentique sur un appel de sous-processus sans interpréteur de commandes, qui ne peut pas donner lieu à une injection mais dont le paramètre est néanmoins signalé comme non validé ;
- deux cas de style et de documentation où le relecteur reste silencieux *parce que l'instruction système le lui demande* : elle proscriit les débats stylistiques et les remarques sur la documentation manquante sauf problème de maintenabilité avéré. Ces cas sondent la limite de cette politique, et testent donc autant la politique que le modèle.

4.7 Déploiement en production

Le service est déployé en conditions réelles : le serveur FastAPI est hébergé sur Railway, la base de données PostgreSQL sur Supabase, et le tableau de bord est accessible publiquement sur un nom de domaine dédié. Le mode Actions est quant à lui distribué sous forme d'image Docker déclarée dans le fichier `action.yml`, ce qui permet à une équipe tierce de l'utiliser en ajoutant quelques lignes à son fichier de workflow.

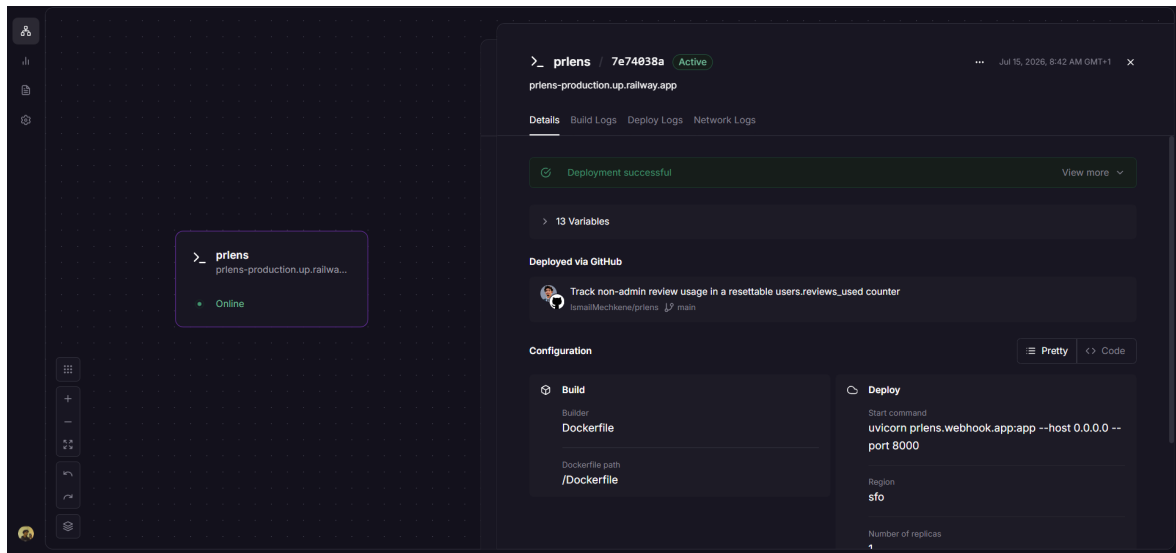


Figure 4.3 – Hébergement du serveur FastAPI sur Railway

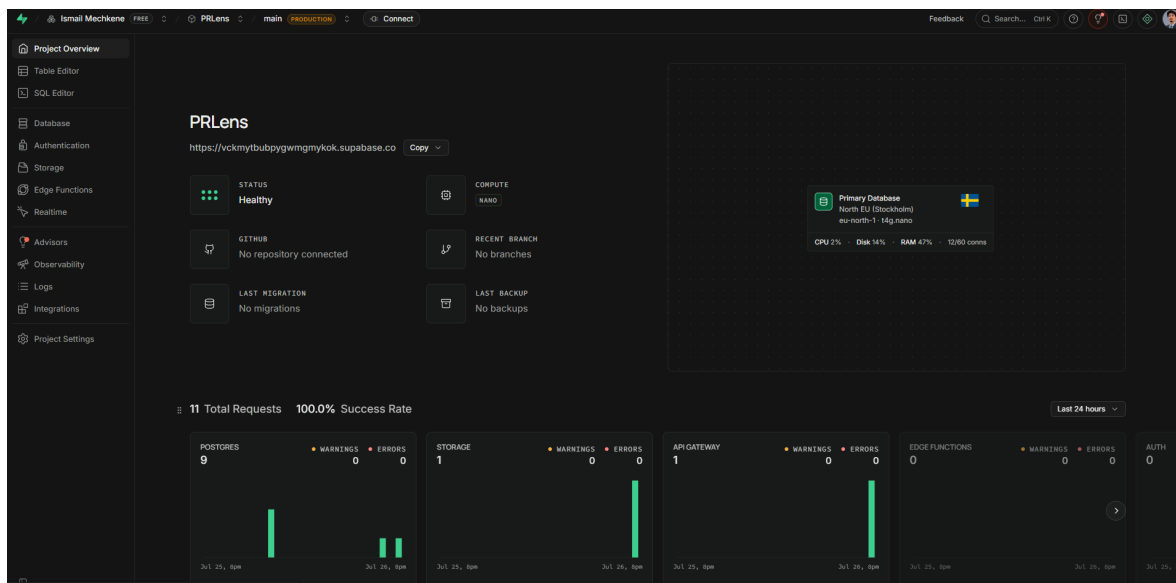


Figure 4.4 – Base de données PostgreSQL hébergée sur Supabase

Conclusion

Ce chapitre a présenté la réalisation de PRLens itération par itération, puis sa double validation : une suite de 147 tests unitaires couvrant 93 % du moteur de revue, et un protocole d'évaluation quantitatif confirmant une pertinence moyenne de 85,7 %, nettement au-dessus du seuil fixé. Les limites identifiées — couverture partielle de la couche API et du client GitHub, faux positifs résiduels sur du code atypique — sont reprises et discutées dans la conclusion générale.

Conclusion générale et perspectives

Ce projet visait à répondre à une problématique clairement identifiée dans le monde du développement logiciel : la revue de code humaine, bien qu'indispensable, est un processus lent, coûteux et inconstant, qui laisse régulièrement passer des problèmes évitables faute de temps ou d'attention disponible. PRLens y apporte une réponse concrète sous la forme d'un système de revue automatisée, capable d'analyser une Pull Request GitHub et de produire, en quelques dizaines de secondes et sans intervention humaine, des commentaires en ligne, un score de qualité et un verdict formel exploitables directement dans le flux de travail existant de l'équipe.

Le travail réalisé a couvert l'ensemble de la chaîne, de l'analyse du besoin métier jusqu'à la validation quantitative des résultats produits : un moteur de revue partagé entre deux modes de déploiement, un modèle de scoring et de verdict déterministe construit sur une source qui ne l'est pas, un tableau de bord web complet permettant de gérer l'ensemble du système sans jamais toucher à un fichier de configuration, et un espace d'administration donnant à l'exploitant la vue globale du déploiement qu'aucun tableau de bord individuel ne peut, par construction, fournir. La validation a été menée sur deux plans : une suite de 147 tests unitaires couvrant 93 % du moteur de revue, et un harnais d'évaluation confirmant une précision moyenne de 85,7 % des revues produites, contre un seuil de 70 % fixé pour le projet.

Ce travail n'est cependant pas exempt de limites, dont certaines sont intrinsèques à l'approche retenue. Le modèle analyse chaque fichier isolément et n'a donc pas de visibilité sur les interactions entre fichiers d'une même Pull Request ; seules les lignes présentes dans le *diff* sont examinées, les problèmes préexistants dans le code non modifié restant hors périmètre par construction ; enfin, le non-déterminisme propre aux modèles de langage peut occasionnellement produire un faux positif sur du code inhabituel mais correct, comme l'a quantifié le harnais d'évaluation. Sur le plan de l'ingénierie logicielle, la couche API du tableau de bord et la logique d'authentification par application GitHub, toutes deux enrichies tardivement, restent en retrait sur la couverture de tests par rapport au reste du moteur. L'espace d'administration est par ailleurs volontairement limité à la consultation : il diagnostique les incidents d'exploitation mais ne les corrige pas, une remédiation réelle passant encore par un accès direct à la base de données.

Sur le plan personnel, ce projet a permis de mobiliser et de consolider des compétences couvrant l'ensemble de la chaîne de développement d'une application moderne : conception d'une architecture logicielle partagée entre plusieurs points d'entrée, intégration avec des API tierces sensibles et leurs mécanismes d'authentification respectifs, modélisation

de données relationnelles, développement d’une interface web complète en React et TypeScript, et surtout mise en place d’un protocole de validation rigoureux ne se limitant pas aux tests unitaires classiques mais mesurant également la qualité du résultat produit par un composant intrinsèquement non déterministe.

Plusieurs perspectives d’évolution se dégagent de ce bilan. À court terme, il conviendrait d’étendre au reste de la couche API et au client GitHub l’effort de test déjà consenti sur le moteur, et de mettre en place un outil de migration de schéma pour fiabiliser les évolutions futures de la base de données — l’ajout des colonnes `role` et `reviews_used` a dû être géré par des instructions `ALTER TABLE` idempotentes au démarrage, une solution qui fonctionne mais ne passe pas à l’échelle de plusieurs évolutions successives. À moyen terme, l’analyse pourrait s’étendre à un **contexte multi-fichiers**, en fournissant au modèle une vue partielle des fichiers liés au sein d’une même Pull Request, afin de détecter des incohérences croisées. Enfin, à plus long terme, le projet pourrait s’ouvrir à d’autres fournisseurs de modèles au-delà d’Azure OpenAI, et enrichir le tableau de bord de fonctionnalités collaboratives : notifications, gestion d’équipes et tableaux de bord par organisation.

Bibliographie

- [1] RAMIREZ, Sebastian. *FastAPI Documentation*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://fastapi.tiangolo.com>
- [2] OPENAI. *OpenAI API Reference*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://platform.openai.com/docs>
- [3] MICROSOFT. *Azure OpenAI Service Documentation*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://learn.microsoft.com/azure/ai-services/openai>
- [4] GITHUB, INC. *GitHub REST API Documentation*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://docs.github.com/rest>
- [5] GITHUB, INC. *Building GitHub Apps*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://docs.github.com/apps>
- [6] PYGITHUB CONTRIBUTORS. *PyGithub Documentation*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://pygithub.readthedocs.io>
- [7] COLVIN, Samuel et al. *Pydantic v2 Documentation*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://docs.pydantic.dev>
- [8] BAYER, Michael. *SQLAlchemy Documentation*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://docs.sqlalchemy.org>
- [9] META PLATFORMS, INC. *React Documentation*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://react.dev>
- [10] OWASP FOUNDATION. *OWASP Top 10*. [en ligne], [référence du 13 juillet 2026]. Disponible sur : <https://owasp.org/www-project-top-ten>
- [11] HARDT, D. (ed.). *The OAuth 2.0 Authorization Framework*. RFC 6749, Internet Engineering Task Force, octobre 2012.
- [12] JONES, M., BRADLEY, J., SAKIMURA, N. *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force, mai 2015.

Appendices

ANNEXE A

Exemple de fichier de configuration `.aireviewer.yml`

```
1 llm_model: gpt-4o
2 min_severity: info
3
4 target_languages:
5   - python
6   - javascript
7   - typescript
8   - java
9
10 excluded_files:
11   - "*.lock"
12   - "*.min.js"
13   - "dist/**"
14   - "build/**"
15
16 reviewers_mapping:
17   security: "IsmailMechkene"
18   quality: "team:senior-devs"
19
20 max_workers: 5
21 approve_threshold: 80
22 changes_threshold: 50
23 large_pr_threshold: 20
```

Listing A.1 – Exemple de fichier de configuration place a la racine d'un depot

ANNEXE B

Exemple de commentaire généré par PRLens

```
1 Severite : critique
2
3 Cle API codee en dur dans le code source. Toute personne ayant
4 acces au depot peut lire ce secret, qui restera en outre present
5 dans l'historique Git meme apres suppression.
6
7 Suggestion : charger la cle depuis une variable d'environnement,
8 par exemple API_KEY = os.environ["OPENAI_API_KEY"].
```

Listing B.1 – Extrait d'un commentaire en ligne genere automatiquement

ANNEXE C

Extrait du modèle de données (SQLAlchemy)

```
1 class Installation(Base):
2     __tablename__ = "installations"
3     __table_args__ = (
4         UniqueConstraint("user_id", "repo_name", name="uq_user_repo"),
5     )
6
7     id: Mapped[int] = mapped_column(primary_key=True)
8     user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
9     repo_name: Mapped[str] = mapped_column(String(150))
10    active: Mapped[bool] = mapped_column(Boolean, default=True)
11    connected: Mapped[bool] = mapped_column(Boolean, default=False)
12    min_severity: Mapped[str] = mapped_column(String(20))
13    approve_threshold: Mapped[int] = mapped_column(Integer)
14    changes_threshold: Mapped[int] = mapped_column(Integer)
```

Listing C.1 – Modèle Installation (extrait simplifié)

ANNEXE D

Liste complète des variables d'environnement

Variable	Nécessaire pour	Description
AZURE_OPENAI_API_KEY	Toujours	Clé d'API Azure OpenAI
AZURE_OPENAI_ENDPOINT	Toujours	Point de terminaison Azure OpenAI
GITHUB_TOKEN	Auth. par jeton	Jeton de repli en l'absence d'application GitHub
GITHUB_APP_ID	Auth. par App	Identifiant de l'application GitHub
GITHUB_APP_INSTALLATION_ID	Auth. par App	Identifiant d'installation de l'application
GITHUB_APP_PRIVATE_KEY_PATH	Auth. par App	Chemin local vers la clé privée (.pem)
GITHUB_APP_PRIVATE_KEY_B64	Auth. par App	Clé privée encodée en base64 (usage conteneur)
GITHUB_WEBHOOK_SECRET	Mode webhook	Secret de signature des livraisons webhook
DATABASE_URL	Tableau de bord	Chaîne de connexion PostgreSQL
SESSION_SECRET	Tableau de bord	Clé de signature des jetons de session
GITHUB_OAUTH_CLIENT_ID	Tableau de bord	Identifiant client de l'application OAuth
GITHUB_OAUTH_CLIENT_SECRET	Tableau de bord	Secret client de l'application OAuth
FRONTEND_URL	Tableau de bord	Origine(s) autorisée(s) pour le CORS et la redirection post-OAuth
VITE_API_BASE_URL	Tableau de bord (client)	Origine du serveur API côté client

Tableau D.1 – Variables d'environnement du serveur PRLens

ANNEXE E

Liste complète des points d'accès de l'API REST

Méthode	Chemin	Objet
GET	/	Vérification d'état du serveur
POST	/webhook	Réception des événements GitHub (signature HMAC vérifiée)
GET	/auth/github	Démarrage du flux OAuth
GET	/auth/callback	Retour OAuth, émission du jeton de session
GET	/api/user	Utilisateur connecté
GET	/api/stats	Statistiques du tableau de bord
GET	/api/repos	Dépôts connectés
GET	/api/repos/{name}	Détail d'un dépôt et de ses paramètres
GET	/api/github/repos	Dépôts GitHub disponibles à la connexion
POST	/api/repos/{name}/enable	Connexion d'un dépôt
DELETE	/api/repos/{name}	Déconnexion d'un dépôt (suppression de l'historique)
PUT	/api/repos/{name}/settings	Mise à jour des paramètres de revue d'un dépôt
PUT	/api/repos/{name}/active	Activation / désactivation de la revue d'un dépôt
GET	/api/reviews	Revue récentes, tous dépôts confondus
GET	/api/repos/{name}/reviews	Revue d'un dépôt donné
GET	/api/admin/stats	Indicateurs globaux du déploiement (administrateur)
GET	/api/admin/users	Liste des comptes (administrateur)
GET	/api/admin/installations	Liste des installations (administrateur)

GET	<code>/api/admin/reviews</code>	Flux global des revues (administrateur)
-----	---------------------------------	--

Tableau E.1 – Points d'accès exposés par le serveur FastAPI